

RESEARCH ARTICLE

WILEY

Unearthing open source decision-making processes: A case study of python enhancement proposals

Pankajeshwara Nand Sharma¹ | Bastin Tony Roy Savarimuthu | Nigel Stanger

Department of Information Science,
University of Otago, Dunedin, New
Zealand

Correspondence

Pankajeshwara Nand Sharma,
Department of Information Science,
University of Otago, 60 Clyde Street,
Dunedin, New Zealand.
Email:
pankaj.sharma@postgrad.otago.ac.nz

Abstract

Good governance practices are pivotal to the success of Open Source Software (OSS) projects. However, the decision-making processes that are made available to stakeholders are at times incomplete and may remain buried and hidden in large amounts of software repository data. This work bridges this gap by unearthing enacted decision-making processes available for Python Enhancement Proposals (PEPs) from 1.54 million email messages that embody decisions made during the evolution of the Python language. This work employs a design science approach in operationalizing a framework called *DeMaP miner* that is used to discover hidden processes using information retrieval and information extraction techniques. It also uses process mining techniques to visualize the processes, and comparative structural analysis techniques to compare different decision processes. The work identifies a richer set of decision-making activities than those reported on the Python website and in prior research work (48 new decision activities, 199 new pathways and 6 new stages). The extracted decision process has been positively evaluated by a prominent member of the Python steering council. The extracted process can be used for process compliance checking and process improvement in OSS communities. Additionally, the DeMaP Miner framework can be extended and customized to suit other OSS projects, such as the OpenJDK project.

KEYWORDS

consensus, decision-making, decisions, open source software development, PEPs, Python

1 | INTRODUCTION

Governance plays an important role in how OSSD communities function.¹ Decision-making (DM) processes are key governance mechanisms that have been shown to be pivotal in smoother functioning and longevity of software communities.² Two main governance-related factors that have a detrimental effect on members in an OSSD community are a lack of process transparency and of consent amongst members on decisions made in the community.³ In particular, researchers have noted that lack of transparency in the decision-making process can alienate members and erode the sense of

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.

© 2022 The Authors. *Software: Practice and Experience* published by John Wiley & Sons Ltd.

community.^{1,4} For example, this issue caused a breakdown in collaboration and raised fairness concerns in the NetBeans community.¹

There is a lack of empirical research that explores OSS decision-making processes.^{5,6} Some OSS projects specifically make their decision-making process explicit, while others do not.⁷ Even when the process details are explicitly made available, the level of detail available within a process can be problematic. Prior work has shown that the high-level decision processes made available in the Python project suffer from a lack of detail⁸ thus rendering them less effective for some stakeholders. So, the challenge with unknown or incomplete processes is that the contributors may not have any understanding or possess only a superficial understanding of one of the key aspects of OSS governance, the decision-making processes.

Duffy et al.⁹ explain this by presenting an example of new members getting involved in OSS projects that use a meritocracy-based governance approach. She notes that since there are no job titles, organizational charts, or design consultancy, it is initially hard for new people to identify the appropriate members who can help them to carry out their tasks. Duffy further explains that members can only achieve this after exercising “patience and careful observation to fully understand the community’s structure, and hierarchy”^{9p. 2}. In addition, even for an internal stakeholder who has been well-integrated within a group, the decision-making procedures may not be readily available in a single document. One might have to sift through several sources of information (e.g., wikis, project web pages, FAQs, and mailing list discussions) and integrate them to understand how decisions are made, which can be both tedious and time consuming.^{10,11} Moreover, even after piecing together different aspects of the process from various documents, the overall process may still be incomplete.⁸ Additionally, the processes that are enacted may be different to the advertised processes.⁷

This work aims to address the above-mentioned gaps by discovering the enacted decision-making processes based on data available in OSS repositories. The Python project has been chosen because of its widespread usage and popularity;¹² its reputation of following good governance practices;¹³ the research interest it has garnered;^{7,8} and because it makes all its data available to the public (e.g., in the form of email archives and commit repositories^{7,14,15}). Additionally, Python operates based on a mixed governance model including elements of both meritocracy and autocracy.¹⁶ Seldom has prior work focused on decision-making of such a mixed governance approach. Also, even when decision models have been studied there has only been a qualitative focus, where high-level models are presented based on interviews and surveys with stakeholders. For example, the work of Mrowka,⁴ presents the decision-making model of the Apache community, which employs a meritocracy approach. Mrowka’s work does not extract the enacted model using a data-driven, bottom-up approach as we undertake in this work. Another reason for studying DM processes in a mixed DM model is that the analysis would highlight the spectrum of DM processes compared to focusing on a project that uses only one DM model.

To this end, in this study we explore methods for mining DM processes from the email archives of OSS projects (the Python developer mailing lists) so that insights of the DM process can be revealed. This work is an extension of our initial work reported in Sharma et al.¹⁶ by presenting more extensive results, with 80% new material. The basis for both our works is to scrutinize the Python decision-making process reported in the literature—those contained within the Figures A1–A3, which are attached in Appendix. A more detailed Python decision-making process is presented using four new tables and five new figures, highlighting aspects not reported earlier. In addition, we further break down RQ2 into sub-parts while the methodology is much more detailed (11 steps in this paper compared to four steps in¹⁶).

The main objective of our work is being able to automatically extract the overall DM process in OSSD unearthing the various activities that take place that are currently hidden in email archives. In doing so, we propose a purposely-built framework called Decision-Making Process (DeMaP) Miner, which can be used to extract DM processes from OSS repositories with similar structure. Thus, this paper makes two key contributions: (i) a methodological contribution in the form of a framework that can extract DM processes from Python repositories, which can be applied to other similar OSS repositories; and (ii) knowledge contributions through (a) extracting the actual DM process used in the Python community and comparing this new process against previously described processes, and (b) highlighting the differences between DM processes used for the three different PEP types.

The article is organized as follows. Section 2 presents the background of the Python OSS project (our case study), contextualizes prior work in this arena, and subsequently presents the research questions. Section 3 details the approach used to extract the Python DM process. Section 4 presents the results and Section 5 evaluates the utility of the extracted process. Section 6 presents the generic DeMaP Miner framework for mining DM processes from OSSD repositories structured similar to Python. Section 7 discusses contributions, limitations, and future work. Section 8 concludes this work.

2 | BACKGROUND AND RELATED WORK ON THE PYTHON OSS PROJECT

We examine OSS DM processes through the case study of Python, a large well-established OSS community. Within the Python community enhancements to the language, that is, a new idea or feature, are made via a *python enhancement proposal* (PEP). There are three types of PEPs:¹⁷

- *Process* PEPs describe the processes used in the Python community, for example, PEP 1 (PEP Purpose and Guidelines) documents the reviewing process used to decide the outcome of proposals.
- *Informational* PEPs elaborate on issues related to Python design or contain guidelines for members.
- *Standards Track* PEPs describe new proposals that enhance the language.

These PEPs are shared with members through the developer mailing list (*python-dev*) and their opinions are solicited and discussed. PEP 1¹⁷ identifies eight main states a PEP document can pass through during the development process (see Appendix Figure A1). Most PEPs end up in the *Final*, *Rejected*, or *Withdrawn* state, while some (e.g., Process PEPs) stay in the *Active* state until revoked or superseded.

There has been some prior work^{7,8} on extracting DM states and creating a process diagram of DM in Python. These studies identified a discrepancy between the “as-is” process and the “should-be” process. Based on historical commit messages for PEP documents, these prior works found five additional main states and showed that the actual process (Figure A2) is more complex than the advertised process (Figure A1). However, the processes these studies extracted are document-centric and state-based. That is, they only consider what happens to the PEP document artifacts by considering just their states (e.g., a document moves from *Draft* to *Accepted* to *Final*), without providing a comprehensive overview of what led to certain decisions. For example, these document centric processes cannot answer questions such as whether a decision was made based on consensus amongst developers, or by pronouncement from a leader despite a lack of consensus. This gap can be bridged by analyzing email messages exchanged between Python core developers, to provide valuable insights into DM activities that lead to state changes in PEPs. For example, analyzing these messages will reveal the underlying consensus building activities, such as voting, that caused a PEP to move from *Draft* to *Accepted*. In other words, this analysis will highlight the unknown DM *sub-states* that occur between the already-known main states.

PEP 1¹⁷ also includes a textual description of the roles of community members in each phase of PEP approval in addition to the diagram shown in Figure A1.¹⁸ Based on this description we manually created a flowchart of the Python DM process (Figure A3). The overall process was divided into five conceptual stages (highlighted in Figure A3 by dashed red outlines): (i) *Idea*, where the proposal is still in its formative stages; (ii) *Draft*, where the proposal is formally drafted after being deemed worth pursuing; (iii) *Community Feedback*, where initial comments towards a proposal are solicited from the community; (iv) *Review*, where the proposal is reviewed by the core developers; and (v) *Review-Update*, where appropriate amendments are made if needed.

Figure A3 encapsulates all that is publicly known about the Python DM process. Similar to the finding of Savarimuthu et al.,⁸ who identified additional states as shown in Figure A2, we contend that vital information about what leads to proposals being accepted or rejected is not included in Figure A3. It is also not known whether Figure A3 accurately depicts how PEP decisions are actually made. Given the limited prior work,^{7,8} one of our goals is to assess whether the DM process shown in Figure A3 is indeed the actual process that is used to enact decisions in Python. To achieve this goal, we need to extract the enacted DM process. Thus, our research objective is to **design and develop an approach that extracts the decision-making process based on the data available in email discussions**. Meeting this research objective enables the answering of the following two research questions (RQs):

- RQ1. What insights can be obtained from the newly extracted decision-making process when compared to previously advertised processes (i.e., those shown in Figures A1–A3)?**
- RQ2. What are the similarities and differences between the process models for the three PEP types (Process, Informational, and Standards Track) when compared to:**
- a. each other**
 - b. the overall DM process model for all PEP types, and**
 - c. the DM process models reported in the literature?**

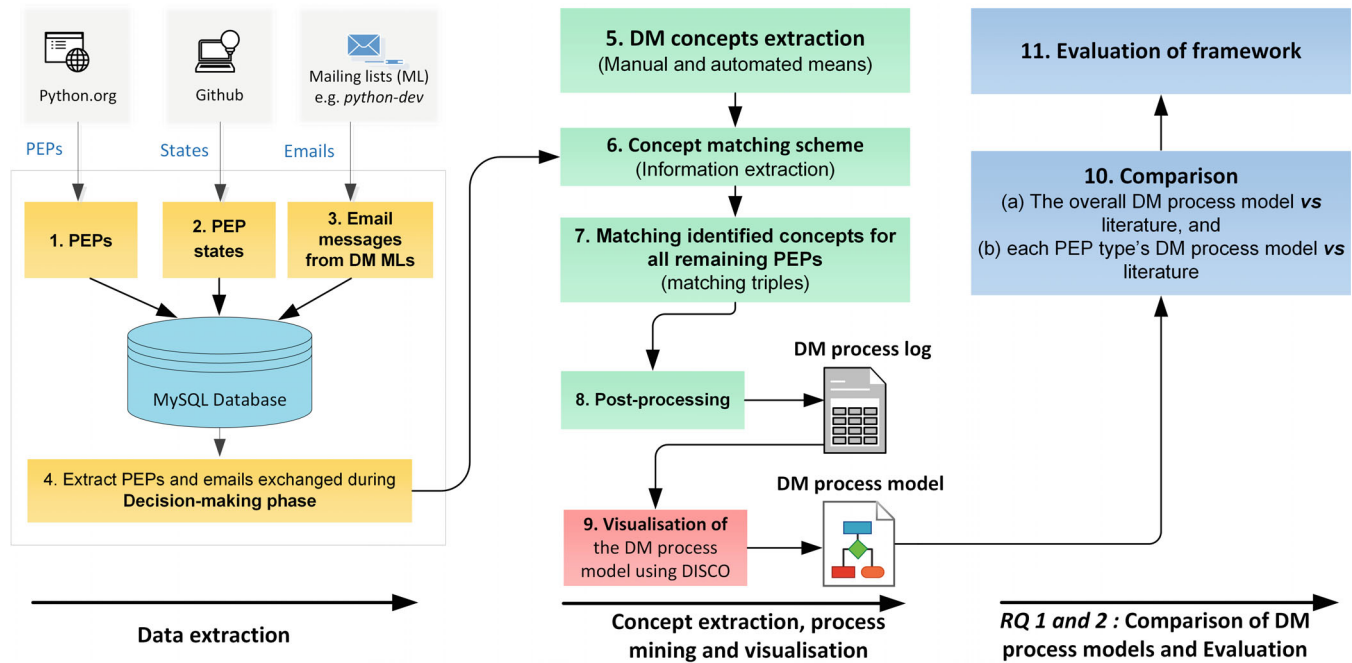


FIGURE 1 Methodology to extract PEP DM processes and answer the RQs

In answering RQ1, a more detailed Python DM process will be highlighted compared to the main states identified in prior work.^{7,8} The detailed DM process will identify sub-structures behind transitions between main states (e.g., how does a PEP get to be accepted or rejected?). The answer to RQ2 will provide a comparative analysis of the extracted process models. The methodology used to meet the research objective of this work is described next.

3 | METHODOLOGY

Our work adopts design science research (DSR), which is commonly used in Software Engineering and Information Systems research.^{19,20} DSR is most-suited for a problem-solving paradigm that focuses on designing a solution to a problem.²¹ DSR research can focus on producing artifacts (e.g., software), theories, or both.²² Our objective here is to develop a software artifact, that is, operationalizing a framework in the form of software that can be used to extract enacted processes.

To create the software framework called Decision Making Process miner (DeMaP Miner for short), we followed the design science research methodology (DSRM).²³ The steps involved: (a) defining the objective of the software, which is to extract enacted processes; (b) developing the software; and (c) evaluating the accuracy and utility of the processes extracted by the software by (i) comparing against manually curated ground truth data, and (ii) seeking feedback from one of the Python steering council members. The steps involved in the construction and the evaluation of the DeMaP Miner software* are presented next.

The methodology for extracting the DM process model for PEPs comprised 11 steps, which are described in detail below and outlined in Figure 1. The dataset used in this study comprised data from three sources: (a) meta-data about each PEP (see step 1 below), (b) state change data for each PEP (step 2), and (c) email discussions drawn from various Python mailing lists (step 3). It spanned the period from the first email posted to the *python-dev* mailing list (March 1995) through to the date the *Benevolent Dictator for Life (BDFL)*, Guido van Rossum, resigned from his post (11th July 2018, just after accepting PEP 572). This enabled us to capture all instances of DM carried out in the Python project using a combination of the meritocracy-based DM principle and the dictatorship-based DM principle.

*The DeMaP Miner tool source code is available at <https://github.com/sharmapn/DeMaPMiner>.

TABLE 1 Emails considered from different Python mailing-lists

Mailing list	PEPs	Number of emails	Emails in DM
<i>python-ideas</i>	279	51,920	8,695
<i>python-dev</i>	454	185,384	36,147
<i>python-checkins</i>	433	156,152	10,290
<i>python-committers</i>	152	5,613	424
<i>python-patches</i>	99	21,319	1,221
<i>python-bugs-list</i>	327	370,770	9,995
<i>python-distutils-sig</i>	97	32,223	5,351
<i>python-3000</i>	133	15,949	3,804
<i>python-list</i>	325	702,810	13,987
TOTAL		1,542,140	89,914

Step 1: Collecting PEP metadata

The first step was to download all 463 PEPs from the above-mentioned period. Among other details, a PEP document contains the PEP number, its authors' details, and BDFL Delegates (if any) who made the final decision on the PEP. We obtained the PEPs from Python's Github repository[†] and stored the extracted metadata in a MySQL database.

Step 2: Collecting PEP state data

Next, we retrieved all the states (out of the 13 possible main states) that each of the aforementioned 463 PEPs transitioned through.⁷ This information was extracted for all PEPs from the commit history in Python's GitHub repository, which includes all versions of every PEP document. For example, PEP 465 traversed through the states *Draft*, *Accepted*, and *Final*.

Step 3: Processing PEP emails

We then extracted from the Python email archives all individual email messages that fell within the above-mentioned period, and stored them in a MySQL database. We also extracted all further email messages pertaining specifically to PEP 572 (the of which was posted on 30th October 2018) so that we had a complete record for all PEPs within the period. We used the following email archives: *python-dev*, *python-ideas*, *python-commits*, *python-checkins*, *python-distutils-sig*, *python-3000*, *python-list*, *python-bugs-list*, and *python-patches*. These nine mailing lists were chosen as they are the leading forums for Python developer discussions, with *python-dev* being acknowledged as the main forum to discuss PEP development. The resulting dataset contained a total of 1,542,140 email messages (see Table 1).

During this step, we also assigned a PEP number to email messages where possible. This was achieved by running a procedure that identified mentions of PEP numbers (e.g., "PEP 13") or titles (e.g., "Python Language Governance") within each message that could be mapped back to a PEP document. This was stored as a field in the database alongside other details of a message (author, date, etc.) so that we could easily extract messages that were related specifically to PEP decision-making.

One challenge with this is that a single message may mention multiple PEPs, and these mentions could contain important information about state changes. For instance, emails in the *python-checkins* mailing list seldom contain message where multiple PEPs are checked in together. To resolve this, we split messages whenever a new PEP is mentioned at the beginning of each paragraph. (Handily, the delimiter "[TAB]" is often used by Python community members in long email messages to separate discussion of different PEPs.) Thus, a message can be associated with many PEPs.

[†]For example, the raw source of PEP 8 can be found at <https://github.com/python/peps/blob/master/pep-0008.txt>.

TABLE 2 DM concepts retrieved from various sources

Source	DM concepts
Literature (7 concepts)	Consensus, lazy consensus, majority, no consensus, no majority, poll, vote
PEP 308 (19 concepts)	Approval to vote, BDFL pronouncement, BDFL pronouncement accepted, BDFL pronouncement rejected, complementary vote, complementary vote results, feedback received, idea approved, idea postponed, no objections, official complementary confirm, poll closed, poll results, positive feedback, requests feedback, vote results, voting method enquiry, voting method suggestion
29 further PEPs (16 concepts)	Acceptance stalled, alternative PEP, assigned PEP number, BDFL asked delegate volunteer, BDFL Delegate accepted PEP, BDFL Delegate allocated, BDFL Delegate reviewed, decided BDFL consensus, idea rejected, member volunteers BDFL Delegate, not enough support, ready BDFL review, ReDraft, request PEP number, reviewed, withdrawn by author
SIGs (6 concepts)	SIG requests feedback, SIG reviewed, SIG consensus, SIG ready BDFL review, SIG BDFL Delegate reviewed, SIG BDFL delegate accepted PEP

We also encountered several messages where neither a PEP number or title was mentioned, but which replied to a message that did. Since these were replies to a message that had already been explicitly associated with PEP decision-making, we assigned them to the same PEP(s) as the message they were in reply to. This produced a dataset of messages related to PEP decision-making that contained 89,914 messages covering 463 PEPs[‡] (see Table 1). Of these, 62 were Informational PEPs, 42 were Process PEPs, and 359 were Standards Track PEPs.

Step 4: Filtering emails exchanged during DM stage

From all the email messages stored in the database, we selected only those that related to PEP decision-making, that is, emails that had successfully been assigned a PEP number and replies to these messages. The remaining emails were deemed not to be related to PEP decision-making.

Step 5: Discovering decision-making concepts

We then explored and analysed Python members' conventions used in email messages containing PEP DM concepts with the aim of identifying fine-grained states, referred to here as *sub-states*, that described transitions between the 13 main states shown in Figure A2. Insights obtained in this step helped us automate extraction of the DM process in later steps. This step comprised four sub-steps.

(a) DM concepts in literature: First, we identified from the literature,^{4,24} a list of seven DM concepts (e.g., *consensus*, *majority*) used in OSSD, including the well-known Apache Project DM model.⁴ These concepts, when embedded in the process diagram, become our sub-states. See the first row of Table 2 for a complete list of these concepts.

(b) Manual analysis of one PEP in-detail: Next, we wanted to quickly grasp how decision-making discussions unfolded in Python, as this would highlight the entities, DM concepts, and conventions used during the development of a PEP. We therefore studied discussions for one highly contested PEP in detail. To find a suitable PEP, we used the concepts identified in the previous sub-step, (and obvious variations, e.g., *voting*, *voted*, etc.) as keywords to search the raw text (flat files) of the entire PEP mailing list archive. We then checked these messages for references to specific PEPs. This enabled

[‡]This is greater than the largest PEP count in Table 1 (454) because some PEPs were discussed only on *python-disutils-sig*.

us to discover PEPs that included these DM concepts in their DM process. We used the DocFetcher[§] desktop search utility to carry out the search process.

We found PEP 308 to be the most contested, as it used the largest number of DM concepts and also had the largest number of associated email messages (i.e., the most discussion). To search for and analyse the remaining DM discussion messages for PEP 308, we built a custom application[¶] to retrieve, display, and navigate through all messages relating to a specific PEP. We manually examined each of PEP 308's 2048 messages, finding a total of 19 additional DM concepts (second row of Table 2).

(c) Manual analysis of 29 additional controversial PEPs: It was unlikely that the discussions of just PEP 308 would contain all possible Python DM terms and concepts, so we manually examined discussions for the next 29 most contested PEPs (identified in the same way as PEP 308 in sub-step 5(b)) in order to identify any further DM concepts. We found five new concepts and associated terms in the first 20 PEPs examined, but found no more in the next nine, and thus stopped our manual analysis at 30 PEPs[#] (comprising 5642 messages), having reached a condition of data saturation.²⁵ These PEPs included 22 Standards Track, five Informational and four Process PEPs (a representative sample from the three PEP types). The DM concepts identified from the additional 29 PEPs are shown in the third row of Table 2.

Special Interest Groups (SIGs): While carrying out sub-step 5(c) we found three PEPs (439, 471, and 3144) that were discussed and decided upon only on the *python-disutils-sig* mailing list rather than *python-dev*. Further investigation revealed that the Python project has many Special Interest Groups (SIGs)²⁶ that support focused efforts concerning specific functional areas of Python. For example, *Distutils-SIG*²⁷ focuses on module distribution utilities. Around 2014, decision-making for PEPs that fell within a SIG's area of expertise was devolved to the SIGs rather than requiring all PEPs to go through *python-dev*, as PEPs discussed there could often “bog down if they address areas where *python-dev* lacks significant expertise”.²⁸ Distutils-SIG is thus a decision-making body in its own right and the PEPs it discusses do not need to be rubber-stamped by the BDFL. Decisions are instead normally made via a BDFL Delegate (mostly Nick Coghlan, one of the prominent members in the Python community). Of the 40 PEPs decided by a BDFL Delegate, 22 (10 Standards Track, six Informational, and six Process) were discussed only within Distutils-SIG, while two further PEPs were discussed in other SIGs. This shows that SIGs are an important element in the Python DM process.

Discussions in a SIG mailing list are similar to those in other mailing lists and thus use similar DM concepts. In order to differentiate DM within SIGs from that through normal channels (*python-dev*), we created a parallel set of SIG-specific DM concepts by adding a “SIG” prefix to the 43 already identified DM concepts. Thus there are two versions of each sub-state, for example, *reviewed* for the normal DM process and *SIG reviewed* for the SIG DM process. We found that six of these SIG-specific DM concepts were used in SIG discussions, as shown in the fourth row of Table 2.

(d) Semi-automated extraction of remaining DM concepts: As the dataset was significant, we could not manually study the entire dataset. To make sure we captured all the remaining PEP DM concepts, we utilised Natural Language Processing (NLP) techniques in the remaining PEP messages.

(i) Email subject analysis: The email subject line seldom contains the main idea of the message, and therefore provides an avenue for capturing DM concepts that may not have been stated in the message body. Triple extraction (discussed shortly) may not capture the concepts from these texts, as these are often not stated as complete sentences, for example, “PEP 308 vote”. Thus, we used the Stanford Parser to extract the part of speech (POS) tags to find the subject and verb from these texts. After removing all PEP numbers, PEP titles, numbers, strings containing numerals, and common message subject prefixes, such as “re”, “fwd”, and “fw”, we used the Stanford Parser to obtain POS tags for each word in the message subject. Words with a *VB*, *VBG*, or *VCN* POS tags were assigned as verbs, and those with an *NN* POS tag as nouns. Once we had all the noun and verb combinations, we manually checked these combinations to see if we could find any more DM concepts. However, we did not find any additional concepts through analysis of the message subject.

(ii) Topic modelling email content for extracting DM concepts: For automatic extraction of DM concepts from Python mailing list discussions, we utilised *topic modelling*. This can reduce the need for manually extracting concepts and extract the remaining concepts (if any). The related terms from topic modelling results could provide us with the DM concepts related to sub-state extraction. The details and results of our topic modelling approach are attached in Appendix

[§]<http://docfetcher.sourceforge.net/en/index.html>.

[¶]A screenshot may be seen at <https://figshare.com/s/1be7d754226c56df2844>.

[#]PEPs 289, 308, 313, 318, 336, 340, 345, 376, 388, 389, 391, 414, 428, 435, 441, 450, 465, 484, 488, 498, 485, 471, 505, 3003, 3103, 3131, & 3144.

(Section A.1). The results did not reveal any new DM concepts and terms beyond those found through our manual analysis.

At the end of this manual analysis step we had unearthed a total of 48 DM concepts (43 relating to the normal Python DM process, and six relating to the SIG DM process). The next step is the extraction of sentences (from their respective PEP email messages) containing these DM concepts from the repository. This will enable us to see patterns in the way these concepts are stated (i.e., linguistic patterns) which can assist in developing a scheme to capture these concepts.

Step 6: DM concept matching scheme

The DM concepts that we were searching for in sentences of OSSD email messages were commonly stated in the form of subject (S), verb (V), and object (O), for example, “BDFL” (S), “accepted” (V), “PEP” (O). In our work, these S,V,O patterns are thus the manifestation of a DM concept that we can capture. We captured all S,V,O patterns corresponding to the DM concepts identified in step 5 from the 30 manually examined PEPs. These captured patterns were then used to extract these concepts from the email discussions for the remaining 433 PEPs (see Table 3). Examples of S,V,O triples that correspond to the concepts of PEP acceptance, consensus, and no majority include “BDFL accepted PEP”, “consensus is reached”, and “proposal lacks majority support”, respectively.

For S,V,O (and hence concept) extraction, we tested three Information Extraction (IE) libraries, ClausIE,²⁹ Reverb,³⁰ and Ollie.³¹ We found that only ClausIE extracted the relevant triples (good triples) that we expected; Reverb and Ollie in contrast, did not. These findings were consistent with previous studies that compared triple extraction using these three libraries.^{32,33} We explain the notion of “relevant triples” as follows. In a study by Rodriguez et al.³³ Reverb and Ollie showed higher precision than ClausIE. However, it was necessary for the IE tool to extract the necessary triples—this relates to the Recall aspect. In the above-mentioned study, ClausIE extracts more triples than Reverb and Ollie (in terms of percentage) and is higher in terms of recall. The work of Bast et al.³² also showed that recall for ClausIE was better than Reverb and Ollie. The results in the study were the same: ClausIE extracted more triples and necessary triples.

ClausIE breaks down each sentence into its basic constituents (clauses), and derives triples from these clauses.²⁹ It uses the Stanford Dependency Parser³⁴ to discover the syntactical structure of an input sentence. Table 3 shows samples of actual concepts (column 2) and corresponding triples (columns 3–5) extracted from PEPs using ClausIE.²⁹ As noted earlier, each concept corresponds to a sub-state. For example, to capture that a BDFL delegate for a PEP (also known as *Benevolent Dictator For One PEP*, or *BDFOP*) has accepted a PEP, the resultant S,V,O triple has “I” as the subject, “accept” as the verb, and an object comprising all the terms “PEP”, “as”, and “BDFOP”, in any order.

TABLE 3 S,V,O triples using triple extraction

Sentence	DM concept	Subject (S)	Verb (V)	Object (O)
“On python-ideas the moratorium idea got fairly positive responses.”	Idea approved	“the moratorium idea”	“got”	“fairly positive responses On python-ideas”
“As the BDFOP for PEP 3151, I hereby accept it for inclusion into Python 3.3.”	BDFL Delegate accepted PEP	“I”	“accept”	“it as the BDFOP for PEP 3151”
“Can you please indicate your vote for or against incorporating PEP 391 into Python?”	Vote	“you”	“can”	“indicate vote”
“Indeed, the wrangling has gone too far already.”	No consensus	“wrangling”	“gone”	“too far”

We also found that DM concepts were also occasionally stated in the message subjects. These message subjects however, are often not well-formed sentences (e.g., “PEP 391–Please Vote!”). To extract DM concepts from these message subjects, we used the Stanford Parser to obtain Part Of Speech tags for each word in the message subject. Thus, instead of triple terms (S,V,O), we identified double terms (verbs plus nouns) that indicated a concept. These doubles did not yield any additional concepts beyond those identified using S,V,O patterns.

Finally, we automatically generated obvious variants of triples already captured, for example, substituting “reject” for “accept”, or “poll” for “vote”, to ensure that all such variants were captured and all triples were mapped to appropriate concepts. This process produced a total of 371 unique S,V,O triples representing the 48 DM concepts found by manual analysis in step 5 (Table 2).

Step 7: Matching DM concepts in PEP discussions

At this stage we were ready to carry out the main DM process extraction. We wanted to determine how and when DM concepts were expressed in email discussions for all PEPs. The keyword list of concepts from step 5 (Table 2) and the S,V,O terms from step 6 (Table 3) were provided as inputs to a module that extracted sentences containing these terms from the 89,914 PEP-related messages for all PEPs in our dataset. We also extracted the timestamp of the containing message for each extracted sentence, which enabled us to identify when a certain DM concept was expressed (e.g., when consensus was achieved). We call these timestamped sentences matching S,V,O patterns *candidate sentences*. These details would be used to create the final process diagram.

To establish whether each candidate sentence indeed expressed a particular DM concept, we again used ClausIE to extract S,V,O triples from these sentences. These new triples were then matched with the 371 unique S,V,O triples curated in the previous step. If any of the triples matched, it established that the sentences exhibited the same DM concept. During this process we omitted candidate sentences where DM concepts were negated by checking for negation terms anywhere within the verbs of the captured triple.

Step 8: Post-processing

We removed repeated DM sub-state captures for the same PEP, as these added duplicate sub-states for the same PEP in the process model. This left us with a total of 9489 confirmed sentences that conveyed the 48 DM concepts identified in step 5.

Step 9: Visualization

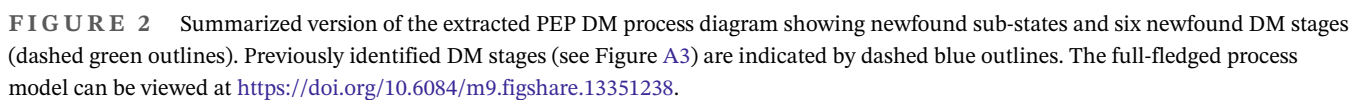
We visualized the results obtained from the earlier steps in the form of process diagrams using the DISCO toolkit³⁵ (e.g., see Figure 2).

Step 10: Comparison

We compared and contrasted the extracted overall DM process with the already known processes (Figures A1–A3) using structural comparative analysis techniques, thus answering RQ1. The results are described in Section 4.1. We then carried out the same analysis for each of the three PEP types, thus answering RQ2. The results are described in Sections 4.2, 4.3, and 4.4.

Step 11: Evaluation

The evaluation was carried out on two levels. First, we evaluated the framework’s ability to capture DM concepts from sentences. Second, we asked a prominent Python core developer to evaluate the overall DM process model that we extracted (Figure 2). The results of the evaluations are described in Section 5.



4 | RESULTS

We begin by presenting the overall PEP DM process model extracted using the DeMaP Miner framework, in Section 4.1. Section 4.2 presents the DM processes for each of the three PEP types and also compares them against one another. Section 4.3 compares the DM processes for the three PEP types against the overall DM process model (Section 4.1). Finally, Section 4.4 compares all of our extracted PEP DM process models against the previous PEP DM process models reported in the literature.

4.1 | Extracted overall process model (RQ1)

In this section, we examine the extracted overall PEP DM process model to see what insights it can provide us compared to previously advertised processes, thus answering RQ1. The overall PEP DM process model is shown in Figure 2. Each node in the process diagram represents an *activity*, which can be either a main state or a sub-state. The edges between nodes represent DM *paths* among states and sub-states. Note that Figure 2 has been summarized for the sake of clarity. The full-fledged process model is more complex and can be viewed on Figshare.^{||}

The goal of RQ1 was to see if the DM process that was implemented differed from the already known processes (Figures A1–A3). We can immediately see that even the summarized version of the extracted process (Figure 2) is more detailed than those presented in Figures A1–A3 (also see Tables 6 and 7). Previous studies in particular focused only on the main DM states of a PEP document.^{7,8} Our approach here has captured additional main states, sub-states (i.e., states between the main states), and DM stages, all of which occur in multiple PEPs. To describe these additional DM sub-states we undertook qualitative analysis of messages belonging to PEPs containing these DM sub-states.

4.1.1 | Main states

Previous research reported 13 main states in the Python DM process.⁷ Our research confirms these states, and a new *Postponed* main state was also discovered. However, because this state only occurred once, we feel it was erroneously used for the *Deferred* main state. This demonstrates that our method is effective in retrieving all main states have been previously reported.⁸

4.1.2 | Additional sub-states and DM stages

The 48 sub-states found by our framework are shown in Table 2. Prior research did not include any of these sub-states,^{7,8} while our manual analysis of the PEP 1 process description (Figure A3) revealed only 17 of them. Thus, our approach has identified 32 new sub-states (Figure 2). As with the manually-derived process diagram in Figure A3, the newly extracted sub-states can be grouped into various high-level stages within the overall DM process (e.g., *Idea*, *Consensus Formation*). The process shown in Figure A3 included five stages (dashed red outlines) that were inferred by the authors (i.e., not explicitly identified in PEP 1). In comparison, our new framework identified nine stages, indicated by dashed blue and green outlines in Figure 2. The blue stages correspond to the same stages in Figure A3. The green stages are either new, or replace those identified previously with more focused versions (e.g., *Consensus Formation*, *Voting*, and *Polling* replace the original *Community Feedback* stage). The original *Review* stage has been absorbed into the *BDFL/BDFL Delegate Decisions* and *SIG BDFL Delegate Decisions* stages.

The six new high-level DM stages are summarized in Table 4. The third column lists the sub-states that appear in each stage, and the second column lists instances of PEPs where these sub-states were identified. We will now discuss these six DM stages in more detail.

^{||}<https://doi.org/10.6084/m9.figshare.13351238>.

TABLE 4 Decision-making sub-states and stages in Python that appear in the extracted process model

Stage	Example PEPs	Sub-states appearing in stage
Consensus formation (informal voting)	Many PEPs	Idea-related: idea proposal, idea approved, idea rejected, alternate PEP, ReDraft Discussion-oriented: favourable feedback, unfavourable feedback, Not enough support Outcome-related: consensus, no consensus, lazy consensus
Polling	255, 308, 318, 428, 488, 572, 3103	Poll, majority, no majority ^a
Voting	279, 308, 391	Formal: approval to vote, vote, majority, no majority ^a Complementary: complementary vote, complementary vote results Discussion: voting method enquiry, voting method suggestion, Voting method discussion
BDFL/BDFL delegate decisions	Many PEPs	BDFL pronouncement, BDFL pronouncement accepted, BDFL pronouncement rejected, BDFL Delegate accepted
SIG consensus formation	439, 471, 3144	SIG requests feedback, SIG reviewed, SIG consensus
SIG BDFL delegate decisions	439, 471, 3144	SIG ready BDFL review, SIG BDFL delegate reviewed, SIG BDFL delegate accepted PEP

^aMajority and no majority appear in both *Polling* and *Voting*.

4.1.3 | Consensus formation

The Python community's DM process mostly revolves around forming a community consensus (upper left in Figure 2). This involves *discussion* and *informal voting*. Discussion occurs during both the early development of an idea before it is formally drafted as a PEP (**idea-related** sub-states in Table 4), and after a PEP is formally drafted and is reviewed by the community (**discussion-oriented** sub-states in Table 4). Our results show that the outcomes of discussions (**outcome-related** sub-states in Table 4) can be classified into *consensus* and *no consensus* sub-states.

We also found instances where community discussion entered a *cycle*, in which certain decision-making sub-states are revisited multiple times. For example, between the *Draft* and *Accepted* main states, a proposal may cycle several times between the *discussion* (and *informal voting* which occurs simultaneously) and *ReDraft* sub-states (the most extreme example involving seven such rounds).³⁶ Similarly, a PEP may cycle through the *discussion*, *no consensus*, and *ReDraft* sub-states, as exemplified by this message by the author for PEP 386 (emphasis added): "As I expected, we did not find a full consensus on the final PEP 386 schema on Distutils-SIG ... So I am proposing to see this PEP *discussed for one more round here*, then rejected or approved by the highest authority:". ³⁷ We found many other cycles in the extracted process model, but these two were the most common ones that involved the newfound sub-states.

4.1.4 | Polling

In the Python community, *Polling* have been taken for a variety of reasons and during various phases of a PEP. These *polls* are undertaken when a PEP is still being finalised during *Consensus Formation* stage, for the entire PEP or for a specific issue within the PEP that has not received unanimous support. Generally, *polls* are commonly undertaken to get an indication of community preferences on a PEP when it is still being completed, and the results of the *poll* have no impact on the final decision, for example, PEP 428.³⁸ Another reason is to gain a sense of public opinion prior to a formal vote, for example, PEP 308.³⁹ For example, during the discussions for PEP 308, the author of the PEP used a *poll*³⁹ to highlight

key ideas to members before *Official Voting* took place. A third reason is to utilise the results directly to determine the outcome of a PEP, in which case only a community majority is required. The *poll* for PEP 488⁴⁰ revealed a majority and the PEP was thus accepted, whereas *polls* for PEP 3103,⁴¹ PEP 255,⁴² and PEP 318⁴³ did not reveal a majority. PEP 3103 was subsequently rejected, PEP 255 was accepted shortly afterwards by BDFL pronouncement,⁴⁴ while PEP 318 went through another round of discussion before being accepted by the BDFL.

In terms of participation, during polling, some PEPs allow non-core-devs to cast vote, however, it is the core developers vote that matters. For instance, non-core-developers also voted in PEP 428.⁴⁵ However, in the *poll* for PEP 572 the only considered votes were those by the core developers. This is evident by the poll results of PEP 572, which are also commented upon on Twitter **. In terms of platforms, most *polls* are conducted in the discussion mailing lists (PEPs 389, 428, 488 and 572). For PEP 3103, there was a quick *poll* by the BDFL at a conference. As mentioned earlier, the *poll* for PEP 488 was announced on Twitter and the actual *poll* was carried out on Google+. The results were formally reported in the Python mailing list. There is even an instance where a webpage was specially created for a *poll* (PEP 318).

While *polls* provides a quick and informal way for anyone in the community to gather preferences, there are instances when an informal approach is insufficient. In terms of usage in making PEP decisions, *Polling* thus falls somewhere between the *Informal Voting* that occurs during *Consensus Formation* and *Formal Voting* as discussed next (i.e., polling is sometimes treated as formal and sometimes treated as informal).

4.1.5 | Voting

Based on our understanding of the Python community, a more formal voting approach is required when it is difficult to decide on the PEP, based only on preferences expressed via *Informal Voting* during *Consensus Formation*, especially when the *Informal Voting* may have spanned multiple different versions of a PEP. If necessary, the Python community uses two voting mechanisms that are more structured in nature. These are *Formal* (or *official*) *Voting* (**formal** in Table 4), and *Unofficial Complementary Voting* (**complementary** in Table 4).

Formal voting is used in two scenarios. The first is for deciding on straightforward proposals. On certain occasions, even after the *Informal Voting* phase, the collective community view may not be certain. PEP 391's author, for example, requested a formal vote after informal voting indicated favourable support for the PEP, but it was unclear whether there was a consensus.⁴⁶ Similarly, PEP279 had previously gotten positive feedback, but an issue remained, so the author held a formal vote before submitting the PEP for BDFL approval. Another purpose of formal voting in the Python community is to facilitate a collective decision on proposals which have several syntaxes as options and when consensus cannot be reached on these during the *Informal Voting* phase. *Formal voting*, like *polling*, seeks a community majority. In the official vote for PEP308, for example, there were eight alternative syntaxes to select from, but the results indicated no majority, and the PEP was later rejected by the BDFL as there was no "overwhelming majority for change".⁴⁷

Unofficial Complementary Voting (UCV) is used to complement the results of a formal vote by conducting a different type of ballot, and using a different method (Condorcet's Method) to tally the results.⁴⁸ Another difference is that while it is clear that the complementary vote is also not a binding vote, when combined with the formal vote "both sets of results... should be more useful to the BDFL than either one would be alone".⁴⁹ For example, PEP 308 included both formal and complementary votes, but neither found a majority, and the PEP was rejected by the BDFL.

Apart from the voting mechanisms, there was also *discussion on voting schemes* (**discussion** in Table 4). With PEP 308, there were discussions in the community about choosing between IRV and the Condorcet methods. The community discussed various voting schemes, including ballot and tally methods (e.g., Condorcet's Method vs. Approval Voting).⁵⁰

In terms of communication platforms for *voting*, both official and unofficial were initiated in the main discussion mailing list, followed by emailing votes (PEP 308) or through a special website and in a newsgroup (*comp.lang.python*)—as in case of PEP 318. The voting *balloting and tally* procedure in the Python community is thorough. Most voting have simple ballot procedures, similar to *polls*, however, in PEP 308 there was a ranked ballot, where members had to specify

**<https://twitter.com/SerhiyStorchaka/status/1014274554452209665>.

their preferences with first, second and third choices. Both the *Official Voting* and the *Unofficial Voting* utilised ranked ballots. There are multiple ways to *tally* votes. These methods include Single Transferable Vote (STV, and its variant IRV, which selects a single winner) and approval voting. A notable aspect is that the voting process is not a winner-takes-all vote but a BDFL-guidance-vote. These votes are not binding, and the final decision is still in the hands of the BDFL.⁴⁸ In summary, in comparison to polling, *formal voting* facilitates a formal approach to ascertain the community's views, which is then mostly used for deciding on PEPs.

We understand that using alternate mechanisms for preference gathering and decision-making in Python may be a result of following the Apache Foundation voting scheme, which prefers lazy consensus and lazy majority. The following comment by a Python member exemplifies this: "... from what I can tell, python-dev is operating in 'majority' or 'lazy' mode most of the times [sic]".⁵¹

4.1.6 | BDFL and BDFL delegate decisions

Our qualitative analysis of the 30 PEPs covered all PEPs that used *Polling*, *Formal Voting*, and only some of the controversial PEPs that used *Informal voting*. From this set of PEPs, we have observed that when making decisions, the collective community view (implies the views of the *core-developers*), based either on the results from consensus formation (i.e., through *Informal Voting*) or the majority view (i.e., through *Polling* and *Formal Voting*) are indeed taken into consideration by the BDFL. The majority of the proposals that used *Polling* and all forms of voting are decided, based on the collective community view. However, as we have outlined before, the preferences of Python community members in these DM schemes are non-binding. The BDFL does reserve the right to make a PEP decision by himself or to overturn a consensus or majority outcome.

PEP 326 is one example of the BDFL making a decision that overrode the community majority formed after the *Informal Voting* phase. The results were 12 for and four against the proposal.⁵² A majority wanted the feature to be implemented, but the BDFL rejected the PEP with detailed reasons: "This has lasted long enough. Python is not a democracy. So here's a BDFL Pronouncement. PEP 326 is REJECTED, because ...".⁵³ The BDFL goes on to list the technical issues he has with the PEP.

In regard to *Polling*, most outcomes were used indirectly by the BDFL while making PEP decisions. PEP 255 was accepted by BDFL, regardless of no majority. PEP 3103 was rejected as there was *no majority*. The lack of a majority in PEP 318 forced it into another round of discussions. The *polls* in PEP 428 and 488 were for a syntax in the PEP, and the preferred syntax in *polls* for both PEPs were chosen as part of the respective PEPs.

In relation to *Formal Voting*, most decisions were made based on their outcomes. The BDFL accepted PEP 279 as more votes were in favour. After many positive votes, the BDFL accepted the PEP 392. Similarly, PEP 308 had a long discussion, where all the DM schemes were used: *Informal Voting*, *Polling*, and *Formal Voting* (both an *Official*, and a *Complementary Vote*). All these schemes could not come up with a community consensus or a clear majority. Thus, the PEP 308 was initially rejected as "*no clear majority*" after the *Formal Voting*. However, after two years, the idea in the PEP resurfaced, and the BDFL eventually accepted the PEP without requiring community preference, again while emphasising the fact that he was still the BDFL: "*It's still my language!:-)*".⁵⁴

We can draw two conclusions in relation to the BDFL's consideration of the collective community view (via consensus or majority). First, while the votes in *Polling* and all forms of *voting* are non-binding, it is evident from the outcomes above that in the Python community, the PEP decisions made by the BDFL are normally made indirectly, based on the outcomes of *Informal Voting*, *Polling* or *Formal Voting*. However, the *Formal Voting* process has a more formal thrust than the *Polling* process. Second, it is evident that while there are some cases where the BDFL overrides the collective community view, in the majority of the proposals we analysed in this section, his decisions reflect the collective view of the community (i.e., of the core-developers).

The next section highlights certain occasions where BDFL Delegates are allowed to make PEP decisions. Based on the findings in this section and the next, the BDFL and the BDFL Delegates make most of decisions on the final outcome of a proposal (accept or reject). Generally the Python community continues into new rounds of *redrafts* and discussions when consensus cannot be reached on a PEP. However, if the community is divided on a proposal and cannot be reconciled, it becomes necessary for the BDFL to intervene and make the PEP decision. For instance, after a disagreement in the community the BDFL accepted PEP 285.³⁷

4.1.7 | SIG consensus formation and SIG BDFL delegate decisions

The activities for *SIG Consensus Formation* were similar to those for PEPs that go through the regular channels (i.e., *python-dev* rather than a Special Interest Group mailing list). We found the following six DM sub-states in the SIG mailing lists: *SIG requests feedback*, *SIG reviewed*, *SIG consensus*, *SIG ready BDFL review*, *SIG BDFL Delegate reviewed*, *SIG BDFL Delegate Accepted PEP*. The presence of the *SIG consensus* sub-state indicates that even in SIGs a collective view is still preferred. Furthermore, the presence of the *SIG BDFL Delegate* sub-states and absence of any of the *SIG BDFL* sub-states (see Table 4) signifies that decisions are made only by the BDFL Delegate assigned to those PEPs. This implies that the BDFL has less influence on PEPs that go through SIGs, and may even imply that the BDFL cannot reject PEPs introduced in SIGs.

We will compare our extracted process model in detail against the three previously published models (Figures A1–A3) in Section 4.4, but even a cursory visual comparison of Figures A1–A3 with Figure 2 shows that our extracted process model includes many more activities (i.e., main states and sub-states), paths, and stages than the prior models. Thus, our extracted process model exposes fine-grained information about DM sub-states and stages that were hidden within the data.

All our above findings regarding different DM aspects related to *Informal Voting*, *Polling*, and *Formal Voting* in the Python community are summarised in Table 5. It also highlights the commonly used balloting and tallying methods, communication platforms, participation, and whether results are binding.

4.2 | DM process models for each PEP type (RQ2a)

In addition to unearthing the overall enacted DM process for Python, our DeMaP Miner framework was also used to discover the DM processes for each of the three PEP types: Process, Informational and Standards Track. The corresponding process models are shown in Figures 3, 4, and 5, respectively. In this section, we compare the similarities and differences of these three process models, thus answering RQ2a.

Table 6 summarizes the stages that appear in the DM process model for each of the three PEP types. We can see that *Consensus Formation*, *BDFL/BDFL Delegate Decisions*, *SIG Consensus Formation*, and *SIG BDFL Delegate Decisions* are in common across all PEP types. This could be reflective of a DM model that is a combination of the meritocracy-based and the dictatorship-based DM principles.

At the same time, there are some differences between the respective DM process models. The DM models for Informational PEPs and Process PEPs are similar, while the DM process model for Standards Track PEPs is much more complex compared to the other two PEP types. The DM process model for Standards Track PEPs adds *Polling* and *Voting* stages (including official and complementary voting, and voting method discussion). These are all geared towards achieving a community majority.

Our findings therefore reveals that the underlying DM processes for the three PEP types are different. This is likely due to the purposes behind each of the PEP types. As Standards Track PEPs deal with the core aspect of the Python community, that is, the evolution of the language, it is more complex to make decisions on them. Consequently they deal with highly technical topics that are prone to many differing opinions. They are also the most common PEP type, comprising about 75% of all PEPs, and are therefore the largest target for decision-making in Python.

In contrast, Informational PEPs are much less common (about 16% of all PEPs) and deal with dissemination of information, so they are less likely to require complex decision-making processes. Process PEPs are even less common (about 9% of all PEPs) and define the Python community's processes. Both types are less likely to attract many differing opinions, which means that the community is unlikely to be presented with multiple conflicting options during the development of Informational or Process PEPs. This lessens the likelihood of an impasse or lack of consensus, thus effectively eliminating the need for DM stages like *Voting* or *Polling*.

4.3 | DM process models for each PEP Type versus overall DM process model (RQ2b)

As shown in Table 6, all four extracted DM process models included the DM stages *Consensus Formation*, *BDFL/BDFL Delegate Decisions*, *SIG Consensus Formation*, and *SIG BDFL Delegate Decisions*. However, only the process model for

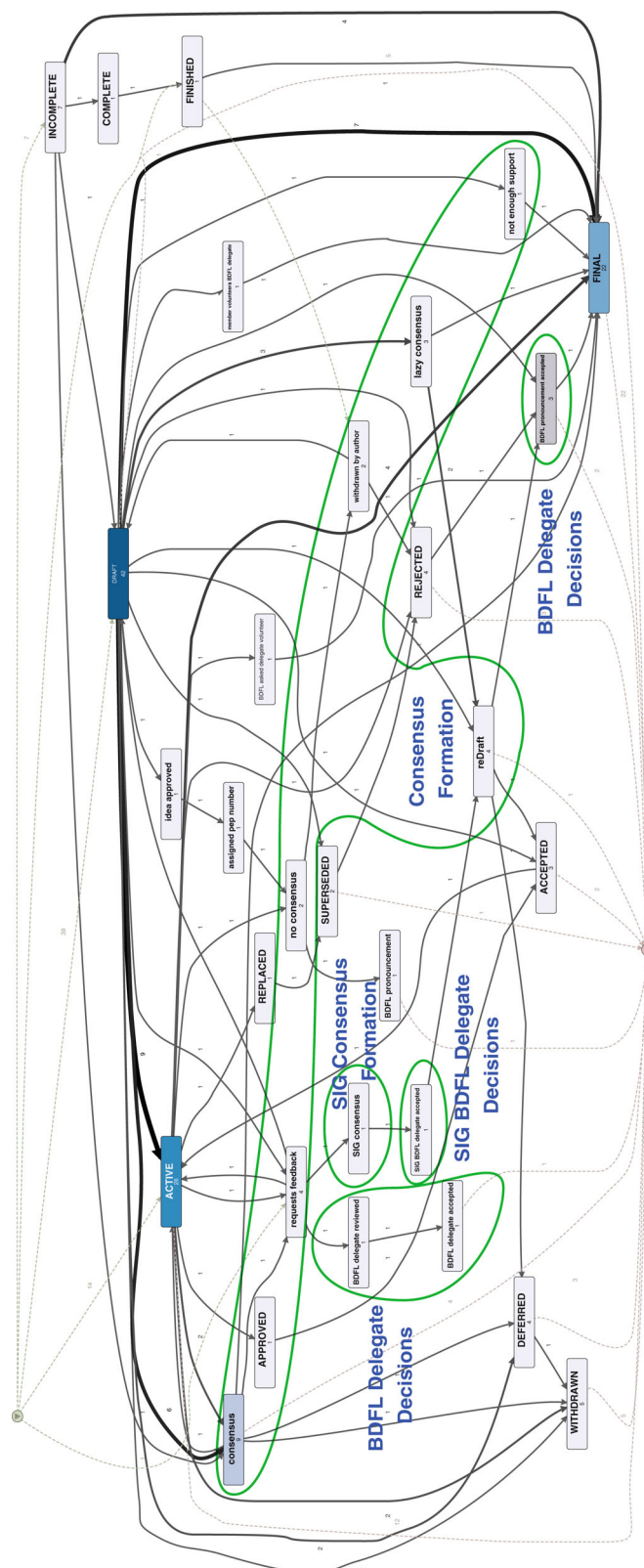


FIGURE 4 Decision-making process diagram for Informational PEPs only. All activities and paths are included. A higher resolution version can be viewed at <https://doi.org/10.6084/m9.figshare.13351445>

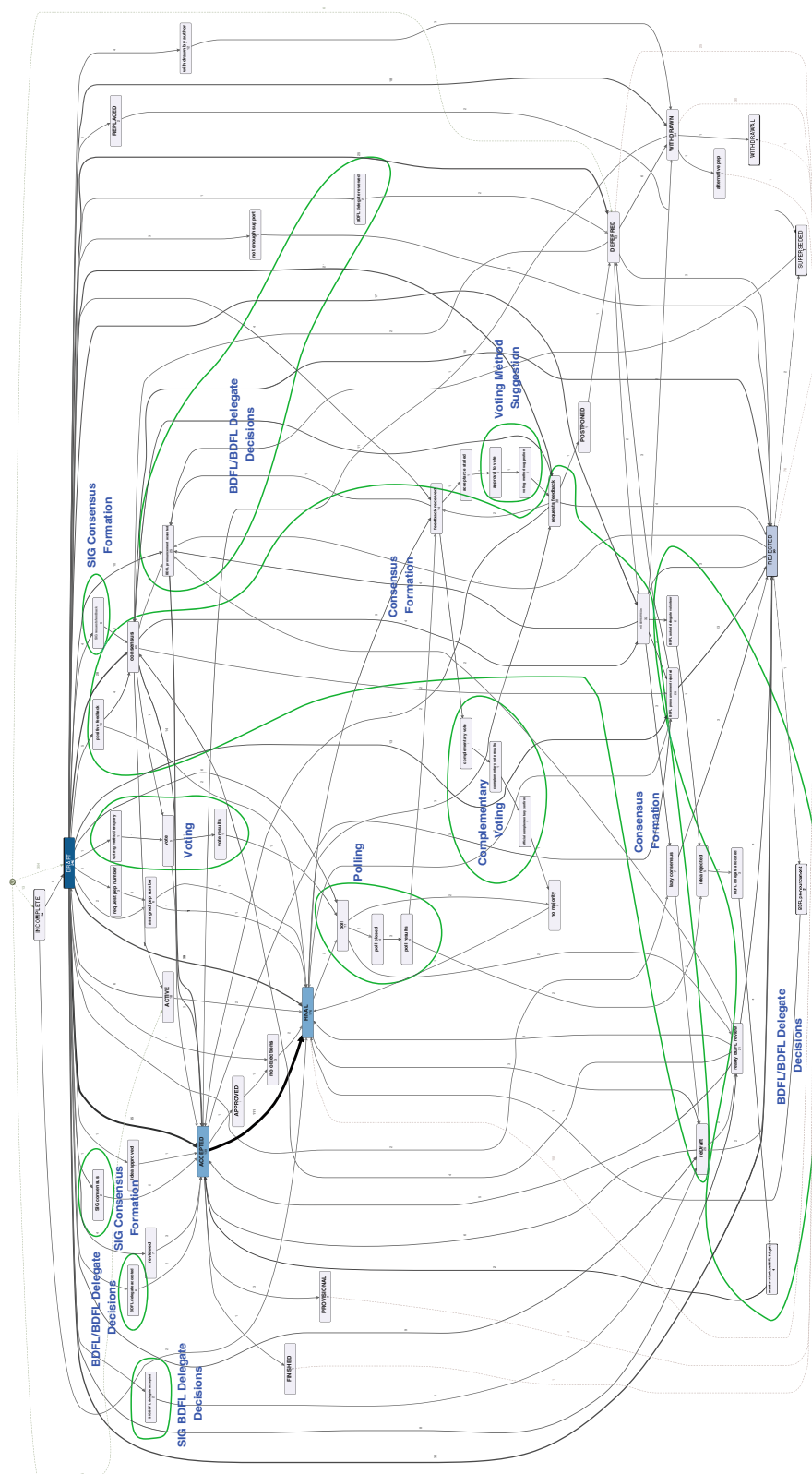


FIGURE 5 Decision-making process diagram for Standards Track PEPs only. All activities are included but only 20% of paths. A higher resolution version can be viewed at <https://doi.org/10.6084/m9.figshare.13351523>

TABLE 5 Comparison of DM aspects during *Informal Voting*, *Polling*, and *Formal Voting* in the Python community

Aspect	Informal Voting	Polling	Formal Voting	
PEPs	Most PEPs	Polls (255, 308, 318, 389, 428, 488, 572, 3103)	Official Voting (279, 308, 391)	Complementary Unofficial Voting (308)
On PEP or part of PEP	Both	PEP as whole (255, 308, 318, 389, 572, 3103) Issue in PEP (428, 488)	PEP as whole (308, 391), Part of PEP (279)	PEP as whole
Reason	Obtain community consensus	Highlight key ideas before main vote (308) Gather community's preference: on PEP (255, 318, 389, 572, 3103)–an issue within PEP (428, 488)	Checking for consensus (391). Gather community preferences on: PEP (308)–an issue (279)	Complement official vote by using a different ballot and tallying method
Platforms	python-dev (most PEPs), python-list, and python ideas. Distutils-sig for specific PEPs (386)	Special website (389), GPlus (488), Mailing list (428, 572), Sending votes to specific email address (308), Special website and on comp.lang.python (318), Poll at conference (255, 3103)	Sending votes to: specific email address (308), python-dev mailing list (279, 391)	Sending votes to specific email address
Participation	Everyone in python-dev. Some contentious PEPs in python-list (e.g., 308)	Everyone in the community via python-list mailing-list (308, 318) Everyone in python-ideas mailing lists (428) Shared publicly on GPlus (488) Core-developers (389, 572) Not specified (255, 3103)	Everyone in python-dev (279) Everyone in the community via python-list mailing-list (308)	Everyone in the community via python-list mailing-list
Ballot	Show support (+1) or disapproval (-1) in mailing-lists	Select preferred option (318, 488)	Hybrid approval/ranked voting (308) Counting responses in mailing-list (279, 391)	Condorcet's method
Tallying	Counting responses or votes.	Standard counting (255, 308, 318, 389, 428, 488, 572, 3013)	Decision based on ML votes (279) Results given to BDFL (308)	Condorcet's method
Results used in DM?	Most (more in next chapter)	No (255, 308, 318, 488, 572) Yes (3103, 389, 428)	Yes (279, 308, 391)	Yes

Standards Track PEPs (Figure 5) shared all six DM stages with the overall DM process model (Figure 2)^{††}. The process models for Process PEPs (Figure 3) and Informational PEPs (Figure 4) both omitted the *Polling* and *Voting* DM stages.

We see a similar pattern for activities (states and sub-states) and event variants (or *pathways*). An event variant is a specific sequence of activities (e.g., a PEP traverses through the following pathway: *Draft* ⇒ *Accepted* ⇒ *Final*). In case of the Python DM process model for each PEP type, they are unique paths from the beginning to the very end of the process. The Process and Informational process models share a similar number of activities and event variants, as do the Standards Track and overall process models. However, the number of activities and event variants in the Process and Informational process models is much smaller than in the other two process models. This further supports the notion that decision-making for Standards Track PEPs is more complex than for Informational and Process PEPs.

^{††}Also see full version at <https://doi.org/10.6084/m9.figshare.13351238>.

TABLE 6 Occurrence of decision-making stages across PEP types

Decision-making stage	Process PEPs	Informational PEPs	Standards Track PEPs	Overall (all PEPs)
Consensus formation (Section 4.1.3)	✓	✓	✓	✓
Polling (Section 4.1.4)	×	×	✓	✓
Voting (all aspects) (Section 4.1.5)	×	×	✓	✓
BDFL/BDFL delegate decisions (Section 4.1.6)	✓	✓	✓	✓
SIG consensus formation (Section 4.1.7)	✓	✓	✓	✓
SIG BDFL delegate decisions (Section 4.1.7)	✓	✓	✓	✓

TABLE 7 Additional activities, pathways, and DM stages in PEP DM for each PEP type

	Process		Informational		Standards track		Overall	
	<i>Keertipati et al.</i> ⁷	<i>Current work</i>	<i>Keertipati et al.</i> ⁷	<i>Current work</i>	<i>Keertipati et al.</i> ⁷	<i>Current Work</i>	<i>Keertipati et al.</i> ⁷	<i>Current Work</i>
Activities^a	6	25	6	30	10	57	11	61
Pathways	10	34	10	36	27	188	36	235
DM Stages	0	3	0	3	0	6	0	6

^aActivities represent both states and sub-states.

4.4 | Extracted process models versus prior models (RQ2c)

Only Keertipati et al.⁷ have previously extracted DM process models for the different PEP types. They found a few additional main states in the enacted DM process model compared to the official DM process model publicized by the Python project. Their study was also limited to main states only, whereas our study reveals that the enacted process model contains many additional sub-states. Table 7 summarizes the differences between our extracted process models and those extracted by Keertipati et al.⁷ This shows that we have found additional activities and pathways, and also identified DM stages that were not previously reported, as discussed in the previous section.

5 | EVALUATION

This section presents evidence both for the robustness of the findings that were used to construct the process diagram in Figure 2, and for the utility of this extracted process diagram. In Section 5.1 we discuss how we evaluated our framework's accuracy at correctly identifying the DM concepts embedded in a sentence. In Section 5.2, we discuss how we solicited stakeholder feedback on the overall extracted decision-making process model.

5.1 | Framework effectiveness

For evaluating our frameworks accuracy in mining DM concepts, three raters (the authors of this paper) judged whether or not sentences labelled by the framework accurately contained the labelled concept (e.g., consensus reached). A total of 200 random sentences were selected from the corpus of all sentences for this evaluation. Each rater also counted the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) produced by the framework for these sentences. The first author (moderator) described how to rate the sentences to the other two raters before the evaluation began, followed by a training session in which examples of selected concepts with matching related sentences were discussed and shown.

As included in Table 8A, Cohen's kappa coefficient (κ) was used to assess inter-rater reliability (IRR) between the three raters (R_1 , R_2 and R_3). This metric assesses the observed level of agreement amongst raters of a certain set of nominal values and accounts for any agreements that may have occurred by chance. Table 8B shows how to interpret the kappa value.

TABLE 8 Cohen's kappa coefficients (κ) for results evaluation

	A. INTER-RATER κ VALUES		
	R_1	R_2	R_3
R_1	1.00	0.78	0.67
R_2	0.78	1.00	0.70
R_3	0.67	0.70	1.00
B. INTERPRETATION			
Range	Agreement		
0.00–0.20	Slight		
0.21–0.40	Fair		
0.41–0.60	Moderate		
0.61–0.80	Substantial		
0.81–1.00	Perfect		

TABLE 9 Evaluation of framework accuracy

TP	121	Precision	0.88
TN	44	Recall	0.87
FP	17	F-score	0.87
FN	18	Accuracy	0.82

We can see that kappa is at least 0.67 across all three raters, indicating substantial agreement. We also computed the Fleiss coefficient⁵⁵ which is widely used for measuring IRR between more than two raters. The Fleiss coefficient was 0.73, indicating good agreement⁵⁶ and is close to the excellent agreement threshold of 0.75. We can therefore be confident that the raters were generally consistent in correctly labelling DM concepts in the 200 sample sentences. We then resolved inconsistencies between the raters by group discussion and consensus. After reconciling these inconsistencies, the agreement level was 1.0 (perfect).

Table 9 shows the results of the framework's evaluation. The TP, TN, FP, and FN scores are shown on the left side of Table 9, while the right side shows the accuracy, precision, recall, and F-score of our framework's recognition of DM states compared to a human (i.e., as agreed by the three raters). The evaluation shows that our framework's performance compares favourably to a human with an accuracy of 82%.

There were 35 instances where labels were captured incorrectly, producing false positives and false negatives. Some of these were due to incomplete sentences and incorrect grammar in the text of messages. The members of OSSD communities discuss similar issues every day, therefore it is easy for them to quickly understand each other's messages even though their messages may not be grammatically correct. Such instances are challenging for an NLP toolkit, however. Other incorrect labels were a result of limitations in ClausIE's triple extraction, which does not always correctly extract all necessary S,V,O triples.

A third source of false positives and negatives is that we were unable to capture all potential negation phrases, such as nuanced negation terms expressing sophisticated human cognition. For example, the following sentence is a false positive for *consensus*, as it actually exhibits both *consensus* and *no consensus* (arguing): 'My impression of the discussion is that the consensus so far is 'maybe' and 'it's too painful', and that's why we're still arguing.'⁵⁷

5.2 | Stakeholder evaluation

In this section, we present the stakeholder evaluation of our results. We aim to answer two basic questions about the Python decision-making process from the viewpoint of a stakeholder involved in the process:

1. Are the *additional* identified DM stages useful for understanding the PEP decision-making process?
2. Is the *overall* extracted DM process model more useful for understanding the PEP decision-making process?

In order to answer these questions, we presented our extracted decision-making process model (Figure 2) to a prominent Python core developer (name withheld to preserve anonymity) in order to confirm whether our extracted PEP process model was accurate, helpful, and contained more information than the description provided in PEP 1 (Figure A3). This person has been a core developer of the Python community since 2004, and has been identified as one of the top Python members in terms of PEP authorship or co-authorship, and being a proxy for the BDFL (i.e., BDFL delegate).⁵⁸ They are also a former Python steering council member.

We provided the stakeholder with the two process diagrams shown in Figures A3 and 2, and asked them to comment on whether Figure 2, at a high level, offered more information to help better comprehend the PEP DM process than Figure A3. We also asked whether Figure 2 was a useful high-level model of the PEP DM process. We highlighted the newfound DM stages enclosed by dashed green outlines in Figure 2 to show the distinctions between the two processes.

Referring to Figure A3, the stakeholder commented: “*Yep, those 6 stages are a solid grouping—the nature of the discussions tends to be quite different depending on where a PEP is in that lifecycle.*” They also agreed that our extracted model shown in Figure 2 indeed supplied more information, and aiding better grasping of the PEP DM process: “*I’ve now reviewed the diagram, and it looks pretty solid to me, and I think does a good job of capturing the different activities than [sic] can be undertaken through the consensus building phase.*” This result shows the utility of both the process model we have discovered, and of the DeMaP Miner framework that was employed to construct the model. It indicates that our extracted DM process model is valid and that the results from our framework are useful.

6 | THE GENERIC DEMAP MINER FRAMEWORK FOR MINING DM PROCESSES FROM OTHER OSSD REPOSITORIES

While the framework has been designed and built for the Python project, extra functionality has been added for the framework to be generic and extensible for extracting the fine-grained DM processes from other similar structured OSSD communities. To this end, this section describe the operationalization of the generic DeMaP Miner framework for extracting DM processes from an OSS project that also uses unique proposal identifiers (i.e., the OpenJDK community).

We will describe the configuration parameters that allow the framework to be extensible using the example of the JDK Enhancement Proposals^{††} to illustrate how the JEP DM process information can be extracted. We describe each stage, including the intermediate output that is produced. Figure 6 presents the high level architecture of the Decision-Making Process (DeMaP) miner framework. All the steps have been described in detail in the preceding sections. In this section, we describe the framework in terms of three functional layers: *data extraction*, *data transformation* and the *visualisation layer*. The process flow in operationalizing the generic DeMaP Miner is illustrated next.

(a) Data extraction layer

This layer provides tools to extract emails and metadata from various data sources that constitute the OSSD community’s communications. Typically, there would be the actual proposals (detailed on the project websites), the mailing list archives holdings discussions of these proposals, and the and state change information of these proposals.

Stage 1: Reading OSSD repositories: There are two steps in this stage. The first is *defining parameters*. The framework includes a configuration file for different parameters in relation to the repository being analysed. The extensibility parameters include (i) *mailing list directories* to read the raw email messages from (ii) *proposal identifier*, for example, ‘JEP’ for JDK Enhancement Proposal, and (iii) *URL* of the database where the mailing list messages will be stored and on which all subsequent queries and analysis will be performed using the framework. The entire database can be re-created using the existing database structure using any MySQL DBMS administration tools (TOAD, for instance). During this step, the database and table names can be modified using search and replace to suit the project (e.g., ‘PEP’ replaced with ‘JEP’) for the framework’s extensibility. The second step is the actual *data storage*. Using the above defined parameters, the process

^{††}<http://openjdk.java.net/jeps/0>.

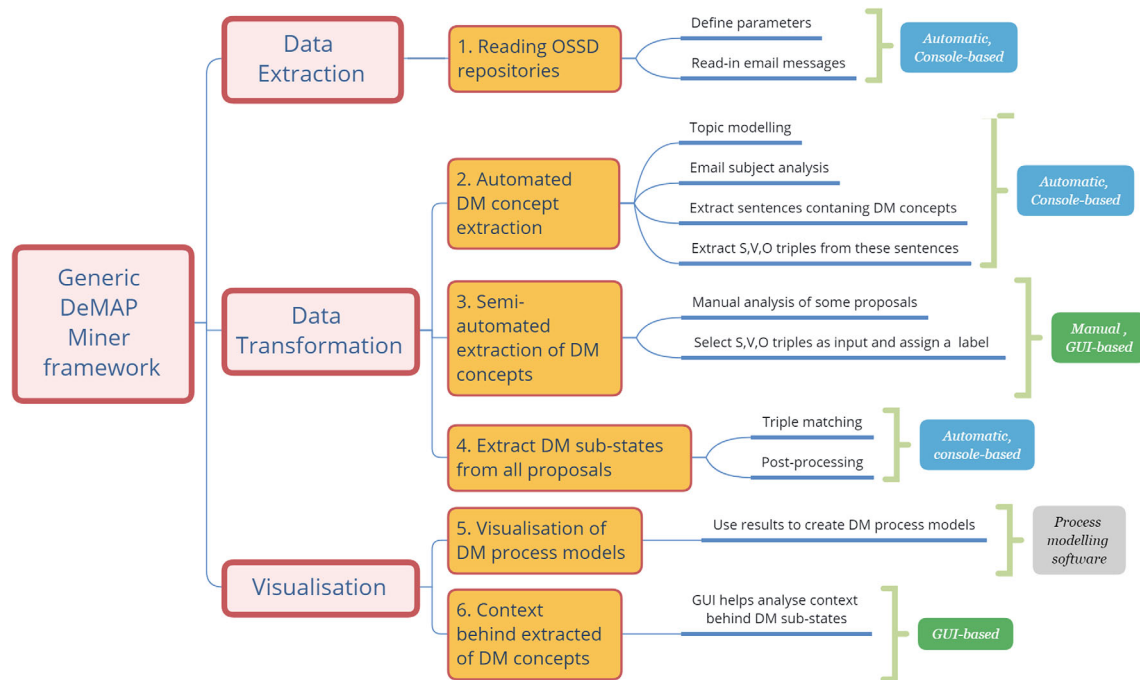


FIGURE 6 The operationalization of the generic DeMaP Miner framework

begins by reading the email messages from the mailing list archives, assigns a proposal number, and stores the messages in database tables. The metadata is read separately and is also stored in a database table.

(b) Data transformation layer

The mailing list data stored in the persistent database is analysed in the data transformation layer. It comprises several modules which implement the phases discussed in the preceding sections. This includes the semi-automatic DM concept extraction scheme, the concept matching scheme, and the post-processing scheme which resolves inconsistencies found in the data.

Stage 2: Automatic DM concept extraction: The stored data can be analysed automatically to retrieve DM concepts (states and sub-states) related to the proposals. Herein, as part of our framework *Email subject analysis* and *Topic modelling* (Section 3) can be used to automatically perform a full analysis of the repository.

Stage 3: Semi-automated extraction of remaining DM concepts and labelling: To get a concrete overview of DM in an OSS project, we propose that a human analyse a small portion (probably 10%) of the dataset to get a feeling of how the DM concepts are expressed. The application we developed to explore the messages for each proposal in the repository, can be used for this task. Once we have all the DM concepts, we can extract all the sentences from the repository which include these concepts. From these sentences we extract S,V,O triples using ClausIE library, as explained in step 6 of Section 3. This task requires human analysis to manually assign labels to triples and create the obvious variations of each triple to match concepts represented by the triples.

Stage 4: Extracting DM sub-states from all proposals: Afterwards, these S,V,O triples are used for DM process mining from the remaining data. We can then match each sentence from every email message related to each proposal and try to capture all sub-states in the dataset. Post-processing will be carried out before the data are ready for visualisation.

(c) Visualisation layer

The application and visualisation layer comprises smaller modules consulting the data stored in the framework's database in order to generate different outputs, allowing the users to carry out empirical studies.

Stage 5: Visualisation of the DM process models: The extracted process log is imported into process modelling software to visualise the DM process models.

- *Navigating proposal messages:* The GUI⁵ mentioned in Section 3 allows users to gain a first and quick understanding of the evolution of each proposal during the DM process.
- *Process mining tools:* Process mining tools are necessary for the visualisation of the extracted DM processes. They include the Disco toolkit.

Stage 6: Understanding the context behind each DM process: A further manual exploration of email discussions relating to DM substates for each proposal will be useful to understand how DM evolved. Figure 7 shows the GUI of the DeMaP Miner that can be used to gain more context for each of the DM sub-states for each proposal. The example shows the extracted DM sub-states for PEP 308. This was a contentious PEP where a number of sub-states had been used, including *official voting*, *complementary voting*, and *polling*—all captured in the bottom left panel. The user can access the message containing the DM sub-states by just clicking on the extracted sub-state. For easier identification, the sentences from which the DM sub-state had been extracted have been underlined, reducing the cognitive load on the user. The particular message shows the context of the *poll*, where the rationale behind it in the subsequent sentence (of the underlined sentence) and that the *poll* was a precursor to the voting exercise.

The framework's generic ability allows it to be plugged into ecosystems which have a unique proposal identifier. Thus, to extract DM processes from other OSS projects, we will first consider projects that are structured similarly to the Python project, for instance the Open JDK community.⁵⁹ Other similar OSS projects include Bitcoin Improvement Proposals (BIPs),⁶⁰ and Debian Enhancement Proposals (DEP).⁶¹ Longer term, we will apply the DeMaP Miner to repositories of OSSD projects that have significantly different DM processes from Python. If we are to analyse some prominent OSSD repositories, for example, Apache, Perl, or Mozilla, more work would be needed to implement our framework to structure their data as they do not have proposal numbers for each proposal (such as a PEP number) discussed in the community. This is discussed in more detail in Section 7.5.4. In this work, we have laid down the foundation for a generic extensible framework for unearthing DM processes from OSSD email repositories. Through some additional work to generalise the tool, a robust framework can be made available.

7 | DISCUSSION

7.1 | A framework for mining DM processes

The overarching research objective of this paper was to **design and develop an approach that extracts the decision-making process based on the data available in email discussions**. In meeting this objective, **our study provides a computational framework that can extract the DM processes that OSS project technical proposals traverse during their life cycle**. The DeMaP Miner framework implements several steps built around information retrieval and NLP techniques (see Figure 1), which aim to automate DM process extraction. Using the framework, we have extracted the detailed Python DM processes from the email repositories, and identified the many sub-states and stages that occur within this process which were previously hidden in email messages.

We have evaluated the results of our framework in two respects. The system evaluation has shown that the framework performs comparably to a human with an accuracy of 82%. Some of the sources of inaccuracy were due to incomplete sentences and incorrect grammar, some were limitations in capturing all possible negation terms from complex sentences, while others were a result of limitations in ClausIE's triple extraction. While we may not have been able to capture all instances of each DM sub-state, we believe that in general, the DM process flow (i.e., the flow between states and sub-states and among sub-states) that we have captured for each PEP type and for all PEPs overall is representative of how this actually occurs.

The stakeholder evaluation of the simplified version of the Python DM process model (Figure 2) was carried out by a Python core developer. This has shown that the extracted process model was indeed useful and provided more information on understanding the PEP DM process, compared to the published process (Figure A3). These evaluations have two implications. First, they validate our extracted PEP decision-making process model. Second, they imply that our DeMaP Miner framework is adequate in extracting DM processes from Python repositories. Thus, it has the potential to be extended into

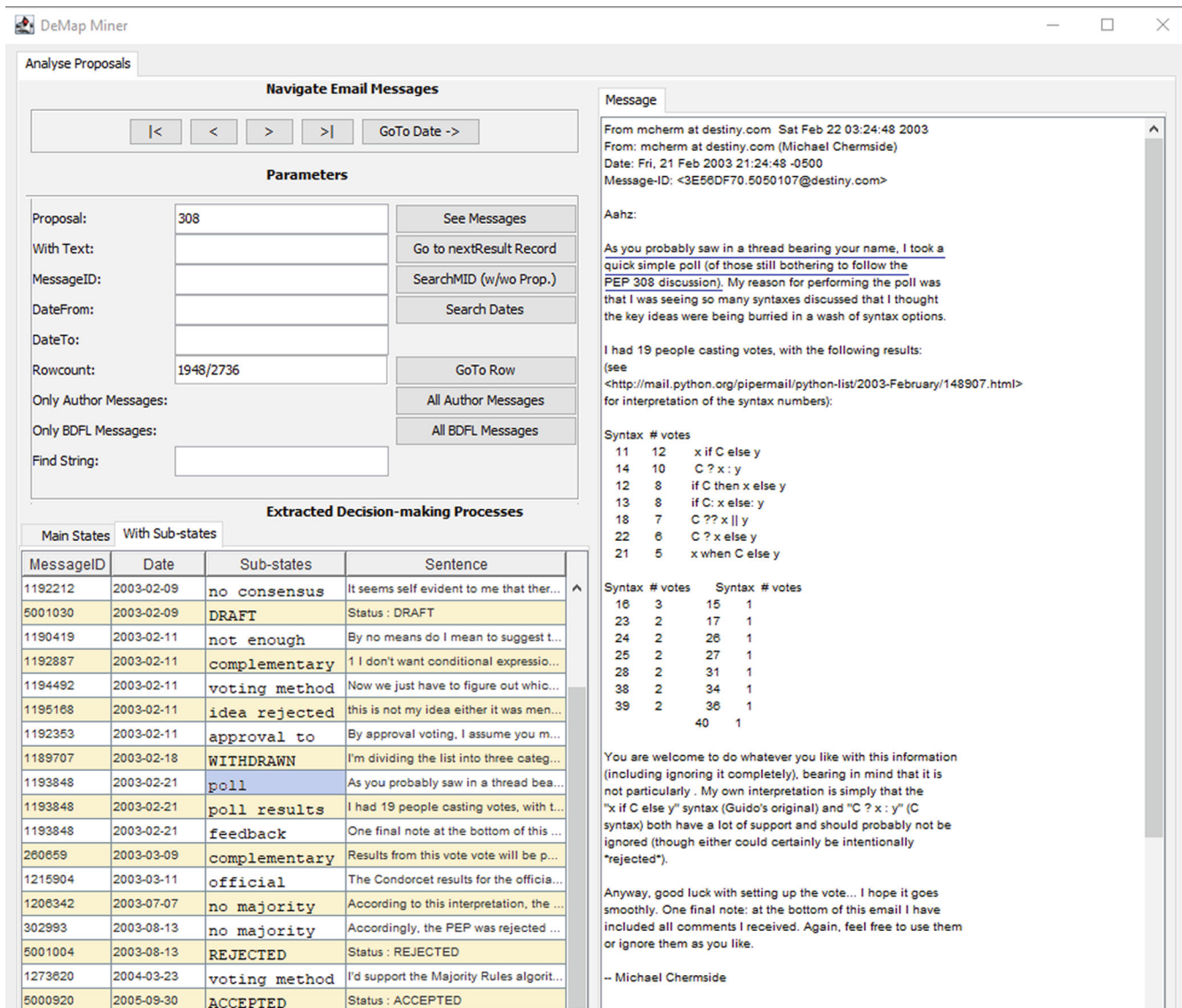


FIGURE 7 The DeMAP Miner GUI shows how a user can navigate to the message containing the extracted sub-states in each PEP and get more context of the DM processes and discussions in each PEP (Note sentences containing the DM sub-states are underlined for easily identifying the main information)

a generic decision-making process extraction tool that can mine DM processes from other OSSD repositories structured similarly to the Python OSSD community.

Examples of other OSS projects that follow the Python model of formal proposals that are openly discussed and debated via email mailing lists include Bitcoin and OpenJDK. The Bitcoin community uses Bitcoin Improvement Proposals (BIPs) to propose enhancements to the Bitcoin infrastructure, while the Java Community Process uses JEPs (JDK Enhancement Proposals) to enhance the Java language. The DeMaP Miner framework can be extended to mine processes from these projects. This will require some reconfiguration to cater for the main high-level DM states in the OpenJDK project, as these are likely to be different from those used in Python.

7.2 | Insights from the enacted overall process (RQ1)

RQ1 focused on the findings of our framework: **What insights can be obtained from the newly extracted decision-making process when compared to previously advertised processes (i.e., those shown in**

TABLE 10 Additional sub-states and stages in Python decision-making

Process Diagram	Source of information					Visualization	
	(Artefact history)	Main states	Sub-states	DM stages	Cycles	Coarseness	Complexity
Figure A1	Official description	8	0	0	0	Coarse	Low
Figure A2	Check-in messages	13	0	0	0	Coarse	Low
Figure A3	PEP 1 description	13	17	5	0	Coarse	Medium
Figure 2	All email messages	14	31 ^a	9 ^b	2	Fine-grained	High

^aFigure 2 is a summarized version of the overall Python DM process model. The full-fledged model has 48 new sub-states.

^bTwo stages from Figure A3 are replaced by six smaller new stages in Figure 2, so effectively found six new DM stages.

Figures A1–A3)? In answering this question, our study makes a knowledge contribution to the area of OSSD DM processes. We have shown that there are additional sub-states and stages in the Python DM process beyond the main states identified in prior studies (as summarized in Table 10). **Our extracted DM process (enacted process) is far richer and more intricate than that advertised by the Python community (Figure A3) and those published previously in the literature.**^{7,8} As noted in the previous section, the stakeholder evaluation of the extracted process also confirmed that the mined process aids a better understanding of PEP decision-making.

The reason that the extracted DM process is more sophisticated than the original PEP 1 process¹⁷ could be due to the PEP DM model's evolution through time. A static model is difficult to sustain in an evolving software development project.^{62,63} In the early days of Python when community was small, the founder made all of the choices, but as it developed, community members' opinions became more significant. (although the majority of final decisions remained in the hands of the project founder, as indicated by our findings). Thus, the process guidelines needed to be flexible in order to accommodate further changes in the project, leading to greater process complexity.

For example, in 2013 the Python community realized that some PEPs could be managed better if discussions were shifted to separate specialized mailing lists, and decisions on them delegated to the experts in the relevant subject areas. This reduced the cognitive workload of the project leader and community members, who may not have the specialized knowledge required to properly engage with these PEPs,²⁸ and allowed for faster decisions to be made. In terms of group decision-making, the fact that PEPs from the *Distutils-SIG* were managed only by members of that group highlights that even under a dictatorship model, Python allowed some expert members to make decisions on PEPs, and does point to a somewhat more collective decision-making approach. More recently, the members of the Python Steering Council confirmed that an objective of the council-based model is to defer decisions on PEPs to “a chosen expert among the core developers (or sometimes outside of that group)”, thus making delegation the default rather than the exception.⁶⁴ This will enable a wider range of people to become involved in shaping the future of Python.

As emphasized in prior research,⁷ the publication of an accurate and thorough decision-making process model, such as ours (Figure 2), can lead to a more “informed” community proposal workflow, where everyone's expectations regarding DM procedures are clear to everyone. When project managers or leaders and junior members have access to such models, they can avoid becoming disheartened (due to a lack of understanding of the different processes involved) or perhaps abandoning the project entirely.⁷ Overall, our model depicts decision-making holistically, allowing for a better comprehension of the overall DM process, which was previously restricted to document-centric state diagrams (Figures A1 and A2) and the manually created process model (Figure A3).

7.3 | DM process models for each PEP type (RQ2)

RQ2 focused on the comparisons of extracted DM process models for each PEP type: **What are the similarities and differences between the process models for the three PEP types (Process, Informational, and Standards Track) when compared to (a) each other; (b) the overall DM process model for all PEP types; and (c) the DM process models reported in the literature?** For each of the three PEP types we extracted the process models using DeMaP Miner and compared them.

First, we found considerable similarity between the three DM process models. The DM stages *Consensus Formation*, *BDFL/BDFL Delegate Decisions*, *SIG Consensus Formation* and the *SIG BDFL Delegate Decisions* were common to all PEP

types. This implies that the Python project does accommodate both the collective community view and the dictator's view when making decisions on all three types of PEP. **Second**, we found that the DM process models for Process and Informational PEPs were similar to each other, whereas the process model for Standards Track PEPs was more complex. In particular, the latter included the additional DM stages *Polling* and *Voting*. This difference arises because Standards Track PEPs deal with the core aspect of the Python community, that is, the evolution of the language, and thus attract more involvement from the core developers.

Third, we observed that the DM process model for Standards Track PEPs was similar to that of the overall enacted process model (i.e., all PEPs). All stages inferred in the overall process model were also present in the process model for Standards Track PEPs. **Finally**, we found that the three extracted PEP-specific DM process models contained more information about decision-making processes than prior work (e.g., Standards Track PEPs had 31 new sub-states, 161 new pathways and six new stages when compared to the work of Keertipati et al.⁷).

Our findings highlight that the DM process for each PEP type varies based on the purpose of the PEP type. Developers new to Python decision-making and researchers who are interested in studying decision processes will benefit from knowledge of the differences between the three processes. Given the similarity between Process and Informational PEPs the DM process for these two could be published as one process model, and the DM process for Standards Track PEPs published as a separate process model. This may be beneficial for developers that need to work on specific PEP types. For example, a core developer working on an Informational PEP does not need to concern themselves with the additional DM stages required for Standards Track PEPs.

We also found that *Distutils-SIG* appeared across all three PEP types. This indicates a healthy and maturing OSS project that has moved away from a dictator being the sole decision-maker, allowing more members to shape the language by making decisions on all aspects of the project.⁶⁴

7.4 | Implications

Using our *DeMaP Miner* framework, we have extracted, identified, and offered a descriptive insight into the 'actual' DM processes of OSS enhancement proposals. The extracted information would be helpful in assisting stakeholders and new community members to understand the underlying governance model. We found additional DM processes (presented as sub-states in this paper), beyond those documented and publicised in the Python project—that considers both: the view of the community and that of the dictator. The key processes we have unearthed include *Consensus Formation*, *Polling* and *Formal Voting*. We also found that consensus formation and the project leader's decisions are common DM phases for all proposals, which is a new finding in the literature.

This study also includes findings on some additional aspects related to DM, based on qualitative analysis, which was missing in the literature on OSSD DM. For the aforementioned three key processes, it reports on how votes are cast (ballot) and counted (tally), and the various communication platforms used. Particularly, it highlights the role of the benevolent dictator in an OSS project with a mixed meritocracy and a dictatorship DM approach. We found that most of the decisions made by the dictator were based on the collective community view. This collective view is mostly the preference of the core developers, gained either through a *consensus* or a *majority*. We have also highlighted a few instances when the project leader overrode the collective view.

We believe that these findings also have implications for practice and theory, which are considered here. Given that transparency in DM has been identified as a critical factor for the success of OSSD projects,³ the incompleteness in previous DM process models noted in our results could be unfavourable to the successful decision-making in Python, and in other OSSD projects where this phenomenon exists. Therefore, based on our findings, we believe that publishing our extracted Python DM process will make the actual DM processes transparent and help stakeholders, junior and future members of the community to quickly grasping how decisions are made in the Python community, especially regarding the more subtle nuances of the process. For example, while the acceptance of PEPs is based on community consensus, there are some instances where the BDFL has approved a PEP despite a lack of consensus. This aspect of the Python DM process is not revealed by examining the previously disclosed DM models (Figure A1), which is state-based (i.e., it contains only main states). The BDFL's capacity to overrule community consensus stemmed from the Python community's monarchist model, although despite this, instances of the BDFL overriding community consensus were uncommon (which is in line with the BDFL's "benevolent" aspect). Again, this is not clear from the originally published process.

There are numerous other DM stages and sub-states that aren't explicit in prior published models (Figures A2 and A3), but are highlighted in our extracted DM model. The inclusion of these finer features in PEP 1 would aid community members in their understanding of DM processes and, ultimately, decision-making. Furthermore, using our DeMaP Miner methodology to other OSSD repositories (e.g., OpenJDK, Linux) would allow us to gain a better understanding of how OSSD communities work in practise, as well as uncover parallels and differences in DM models between projects. It could potentially pave the path for a broader OSSD DM model.

7.5 | Limitations, threats to validity and future work

This section describes the limitations and threats to validity of our work, as posed by our research method, and the actions that we took to mitigate them.

7.5.1 | Limitations

This study developed organically out of a need to answer specific research questions based on the available data set about PEP development. As such, it has several limitations which we aim to address in future work.

First, while carrying out our exploratory analysis of states, sub-states, and DM stages in the Python DM process, we manually analysed email messages relating to only 30 PEPs rather than all PEPs (which would have been prohibitively time-consuming). However, we reached data saturation (no new concepts were found) after analysing messages for 20 PEPs. We therefore believe that we have captured all the important concepts in Python DM. The positive stakeholder evaluation from the Python steering council member supports this. However, we concede that we may have missed some Python DM concepts.

Second, some DM concepts may not have been captured as our framework relies on the triple extraction capabilities of information extraction libraries. Although these libraries do not require any background knowledge, they do not always extract all the subjects, verbs, and objects from sentences. We believe this is a limitation for any NLP research, however, not just ours. For this reason, we place more emphasis in our work on comparisons of DM stages rather than individual sub-states. *Third*, while Python email archives are the official communication repository of the Python OSSD community, we have noted that developers sometimes opt for a faster means of interaction, such as a chat-based system (e.g., the freenode IRC network), social networking sites (e.g., Twitter), or face-to-face interactions. Unfortunately, transcripts of these communications are not available for us to extract more useful information about decision-making.

Fourth, a limitation of the process mining techniques so far developed is the huge number of potential activities (i.e., states and sub-states) and paths between these activities. In the fully fledged version of our overall DM process diagram for all PEPs, there are 61 activities and 235 different paths between these activities. Figure 2 presents a summarized version of this process model, showing only 20% of paths for readability. This enabled us to visualize a structured process model for all PEPs. However, we are mindful that the application of our framework to a different OSSD repository may produce a larger set of activities and paths, leading to a highly complex process diagram. Process mining is a visual technology, most usefully applied in close collaboration between (in our case) OSSD decision-makers and process mining experts. To resolve and comprehend this large amount of information, we need to involve domain experts at regular intervals in summarizing these process models in order to develop the most useful representations and visualizations of DM in these communities.

7.5.2 | Threats to internal validity

As discussed in Section 5.1, we evaluated the inter-rater reliability of three human raters at identifying sentences that contained DM concepts. The Cohen's kappa coefficient (before reconciliation) was at least 0.67 across all three raters, indicating substantial agreement. Similarly, the Fleiss coefficient was 0.73, indicating "good" agreement.⁵⁶ After reconciling the inconsistencies, the agreement was 100%. This was then used as the baseline for calculating the accuracy of our framework at classifying DM concepts (82%). We are therefore confident that this is a true reflection of our framework's capability.

As time goes by, governance models in an OSSD community will inevitably change. For example, the departure of prominent decision-makers will change how decisions are made in the future (e.g., the BDFL's resignation in 2018). To capture the new DM models, new discussion archives will need to be extracted, stored and analysed, which is a simple task in our framework. In this way we hope to develop a longitudinal analysis of OSSD DM processes. Even in the face of such change, our model is still of utility. Feedback from the stakeholder evaluation noted that recent changes to the Python DM process still fit within our extracted process: "Even the changes we made following the Steering Council election aren't likely to fundamentally alter it—they'll just change the sequence a bit, such that a BDFL-Delegate is typically selected earlier in the process."

7.5.3 | Threats to external validity

The main threat to external validity is the generalization of our DeMaP Miner framework to other OSS projects. In this work, we are limited to the investigation of a single case study involving Python, due to the sheer volume of information that needs to be manually analysed. In the future, we intend to study other OSS projects in order to get a broader understanding of their decision-making processes and reasons behind decisions. We are particularly interested in the OpenJDK project, whose DM structure is inspired by Python, but also in other communities that have a BDFL, like Linux and Perl. For this purpose we have made the DeMaP Miner framework as generic as possible, with parameter files enabling variable values such as the proposal identifier (e.g., "PEP", "JEP") to be configured.

7.5.4 | Future research directions

This section outlines avenues for future research in relation to finding out more on people and processes during DM in OSSD.

- **Extracting DM sub-states from other OSSD repositories:** In the future, we also plan to extend the use of our *DeMaP Miner* framework to extract fine-grained DM processes, which lie hidden in large email repositories of other OSS projects, including those that employ community-based DM models such as PostgreSQL, and those with monarchist DM models such as Linux and Perl. These investigations will help us understand how decisions are made in these projects and to gain a broader understanding of the DM process in OSSD in general. We also intend to study how DM processes change as projects mature. As discussed in literature,¹ dictatorship-based OSS projects in time tend to embrace more open and democratic systems (e.g., Linux).

For these purposes, we intend to generalise the DeMaP Miner framework so that it works in other OSSD repositories, starting with the OpenJDK community. The OpenJDK DM process is based on that of Python, but with additional states.⁶⁵ Regardless, first, a detailed manual study of emails belonging to a portion of the OpenJDK proposals to grasp how the DM processes are expressed will be required.

- **Structuring unstructured OSSD repositories for analysis:** When applying the DeMaP miner framework to other OSSD repositories, we would certainly encounter OSS projects which do not have proposal numbers. Examples are Apache, Perl and Linux, where proposals are referred to by their title, not a number. Thus, it would be more difficult to trace discussions relating to a proposal. A future study could be carried out using computational techniques such as clustering to identify similar proposal titles. *Topic matching* approaches can also provide an approximate judgement of these.

8 | CONCLUSION

The objective of this work was to unearth hidden open-source decision-making processes via a case study of Python comprising 1.54 million emails. To this end, we developed and propose the *DeMaP Miner* framework, and in doing so our work offers two types of contributions.

First, our work offers several *knowledge contributions* based on using the framework on the Python repository data. In total four DM processes were discovered using this framework: the overall extracted process and one process for each of the three PEP types. The newly discovered enacted process contained 48 new decision sub-states, 199 new pathways, and 6 new DM stages (e.g., *Voting*, *Consensus Formation*) in the overall process. This shows that the enacted process is more complex than the publicized process. We believe that publicizing our extracted Python decision-making process will benefit the Python community by making the actual decision-making processes more explicit and transparent. It will help stakeholders, junior members, and future members to quickly grasp how decisions are made in the Python community, thus promoting greater cohesion within the community and helping to reduce turnover of project members. We also believe our knowledge contributions will help academia and practitioners in the enhanced understanding of people and processes in OSSD communities during DM. In particular, we propose that the Python community should make the extracted (fine-grained) DM process model available to the public.

Second, the developed *DeMaP Miner* framework that was used to achieve this objective is a *methodological contribution*. The generic *DeMaP Miner* framework which can be used to extract DM processes, from other OSS projects structured similar to Python. We can plug in email repositories of these projects to this framework and, with minor changes in configuration, mine these information. This undertaking will allow for a broader understanding of the DM processes in OSSD, in general. We evaluated the accuracy of the *DeMaP Miner* framework at identifying Python DM concepts and found that it was comparable to that of human raters, with an overall accuracy of 82%. We also undertook a stakeholder evaluation with a Python Steering Council member, who confirmed that the enacted process that we extracted was similar to the actual DM process used. Based on the positive results of these two evaluations, we believe that our framework is effective at unearthing hidden DM processes. With appropriate modifications, the framework can be used to extract PEP-like DM processes in other communities, such as OpenJDK's Java Enhancement Proposals (JEPs). This will be of great benefit to researchers interested in studying OSSD decision-making processes.

AUTHOR CONTRIBUTIONS

Pankajeshwara Nand Sharma: Conceptualization (lead); data curation (lead); software (lead); methodology (equal); visualization (lead); validation (equal); writing – original draft (lead). **Bastin Tony Roy Savarimuthu:** Methodology (equal); supervision (equal); writing – review and editing (equal); validation (equal). **Nigel Stanger:** Supervision (equal); methodology (equal); writing – review and editing (equal); validation (equal).

ACKNOWLEDGMENT

The first author would like to thank Russell Education Trust for the scholarship he received. Also, part of this work was supported by the Otago Business School research grant awarded to Dr Nigel Stanger and Dr Tony Savarimuthu. The first author also thanks the University of Otago for the post graduate publishing scholarship to develop the paper. Open access publishing facilitated by University of Otago, as part of the Wiley - University of Otago agreement via the Council of Australian University Librarians.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Pankajeshwara Nand Sharma  <https://orcid.org/0000-0001-9159-8332>

REFERENCES

1. Siobhán O'M, Fabrizio F. The emergence of governance in an open source community. *Acad Manage J*. 2007;50(5):1079-1106.
2. Ron G, Gabriel Richard P. *Innovation Happens Elsewhere: Open Source as Business Strategy*. Morgan Kaufmann; 2005.
3. Walt S, Chris J. *Governance in Open Source Software Development Projects: Towards a Model for Network-Centric Edge Organizations*. California University Irvine Institute for Software Research; 2008.
4. Rafał M. Decision-making in the process of implementation of open source projects. *Bus Sys Econ*. 2015;2(2):39-49.
5. Yeliz EU, Kangning W, Kevin C. Decision-making processes in community-based free/libre open source software-development teams with internal governance: an extension to decision-making theory. *Commun Assoc Inf Syst*. 2020;46(1):20.
6. Thapa R, Vidolov S. Evaluating distributed leadership in open source software communities. In: *Proceedings of the 28th European Conference on Information Systems*, 2020.

7. Keertipati S, Licorish SA, Savarimuthu BTR. Exploring decision-making processes in python. In: Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering, 2016, 1-10.
8. Roy SBT, Khanh DH, Sherlock L, Smitha K, Daniel A, Ghose Aditya K. Process compliance in open source software development—A study of Python enhancement proposals (PEPs). *European Conference in Information Systems*, 2016, 48.
9. Duffy M. Fedora: A case study of design in a FLOSS community, <http://duffy.fedorapeople.org/presentations/chihop/duffy-flossdesign6.pdf2009>.
10. Cornelia M, Rainer W. Classification of design decisions—an expert survey in practice. *European Conference on Software Architecture*. Springer; 2013:130-145.
11. Xueying L, Peng L, Zengyang L. Automatic identification of decisions from the hibernate developer mailing list. In: Proceedings of the Evaluation and Assessment in Software Engineering, 2020, 51-60.
12. Popularity of Programming Language Index . PYPL Popularity of Programming Language. <https://pypl.github.io/PYPL.html2020>
13. Wang J, Shih Patrick C, Yu W, Carroll John M. Comparative case studies of open source software peer review practices. *Inf Software Technol*. 2015;67:1-12.
14. Guzzi A, Bacchelli A, Lanza M, Pinzger M, Van Deursen A. Communication in open source software development mailing lists, Proceedings of the 2013 10th Working Conference on Mining Software Repositories (MSR), IEEE, 2013, 277–286.
15. Hoa Khanh D, Roy Tony Savarimuthu B, Avery D, Ghose A. Mining software repositories for social norms. Proceedings of the 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE), IEEE, 2015, 627–630.
16. Pankajeshwara S, Roy SBT, Nigel S. Mining decision-making processes in open source software development: a study of python enhancement proposals (PEPs) using email repositories. In: Proceedings of the Evaluation and Assessment in Software Engineering, 2020, 200-209.
17. Barry W, Jeremy H, David G, Nick C. PEP 1–PEP Purpose and Guidelines. <https://www.python.org/dev/peps/pep-0001/2013>
18. Pankajeshwara S, Roy SBT, Nigel S, Licorish Sherlock A, Austen R, eds. Investigating Developers' Email Discussions during Decision-Making in Python Language Evolution, ACM, 2017, 286–291.
19. Wieringa RJ. *Design Science Methodology for Information Systems and Software Engineering*. Springer; 2014.
20. Per R, Emelie E, Margaret-Anne S. *The Design Science Paradigm as a Frame for Empirical Software Engineering*. Springer; 2020:127-147.
21. Alan H, Samir C. *Design Science Research in Information Systems*. Springer; 2010:9-22.
22. Richard B, Abayomi B, Shirley G, Alan H, Matti R. Design science research contributions: finding a balance between artifact and theory. *J Assoc Info Sys*. 2018;19(5):3.
23. Ken P, Tuure T, Rothenberger Marcus A, Samir C. A design science research methodology for information systems research. *J Manag Info Sys*. 2007;24(3):45-77.
24. Karl F. *Producing Open Source Software: How to Run A Successful Free Software Project*. O'Reilly; 2005.
25. Faulkner Sandra L, Trotter SP. Data saturation. The international Encyclopedia of communication research methods, 2012, 1–2.
26. Python . Python Special Interest Groups, <https://www.python.org/community/sigs/2020>
27. Python. Python Distutils-SIG: Introduction. <https://www.python.org/community/sigs/current/distutils-sig/2020>
28. Jonathan C. Python packaging: playing well with others. <https://lwn.net/Articles/580399/2014>
29. Luciano DC, Rainer G. ClausIE: Clause-based open information extraction, ACM, 2013, 355–366.
30. Anthony F, Stephen S, Oren E. Identifying relations for open information extraction. Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2011, 1535–1545.
31. Oren E, Bart Robert E, Mausum, Schmitz Michael D, Soderland Stephen G. Open language learning for information extraction US Patent Application 14/083, 3422014.
32. Hannah B, Elmar H. Open information extraction via contextual sentence decomposition. Proceedings of the 2013 IEEE Seventh International Conference on Semantic Computing, 2013, 154-159.
33. Rodríguez Juan M, Merlino Hernán D, Patricia P, Ramón García-Martínez. Performance evaluation of knowledge extraction methods. *Trends in Applied Knowledge-Based Systems and Data Science*. Springer; 2016:16-22.
34. Christopher M, Mihai S, John B, Jenny F, Steven B, David M. The Stanford CoreNLP natural language processing toolkit. Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 2014, 55–60.
35. Günther Christian W, Anne R. Disco: discover your processes. *CEUR Workshop Proc*. 2012;940:40-44.
36. Aahz M. [Python-Dev] file() vs open(), round 7. <https://mail.python.org/pipermail/python-dev/2005-December/059073.html2005>
37. Tarek Z. [Distutils] Proposing PEP 386—a standard version scheme. <https://mail.python.org/pipermail/distutils-sig/2009-December/014875.html>
38. Antoine P. PEP 428—The pathlib module—object-oriented filesystem paths. <https://www.python.org/dev/peps/pep-04282019>
39. Michael C. PEP 308: Results of “Helping Aahz set up the ballot”. <https://mail.python.org/pipermail/python-list/2003-February/212542.html>
40. Brett C. [Python-Dev] PEP 488: elimination of PYO files. <https://mail.python.org/pipermail/python-dev/2015-March/138777.html>
41. Mark L. Case Statements <https://mail.python.org/pipermail/python-list/2016-March/855566.html>
42. Warsaw Barry A. [Python-Dev] Suggested amendment to PEP 255. <https://mail.python.org/pipermail/python-dev/2001-June/015487.html>
43. Josiah C. [Python-Dev] PEP 318: Can't we all just get along? <https://mail.python.org/pipermail/python-dev/2004-August/048048.html>
44. Guido R. [Python-Dev] PEP 255 - BDFL Pronouncement: ‘def’ it stays. <https://mail.python.org/pipermail/python-dev/2001-June/015500.html>

45. Python Mailing List Discussions: [Python-ideas] PEP 428: poll about the joining syntax. <https://mail.python.org/pipermail/python-ideas/2012-October/016585.html>
46. Vinay S. [Python-Dev] PEP 391–Please Vote! <https://mail.python.org/pipermail/python-dev/2010-January/097335.html>
47. Raymond H. [Python-checkins] python/nondist/peps pep-0000.txt, 1.249, 1.250 pep-0308.txt, 1.19, 1.20. <https://mail.python.org/pipermail/python-checkins/2003-August/037477.html>
48. Norman P. RESULTS: Complementary PEP308 Ternary Vote <https://mail.python.org/pipermail/python-announce-list/2003-March/002102.html>
49. Norman P. Revised ANN: Re: complementary PEP308 Ternary VOTE. <https://mail.python.org/pipermail/python-list/2003-March/224392.html>
50. Dave B. PEP 308 vote type (was Re: Update to PEP308: if-then-else expression). <https://mail.python.org/pipermail/python-list/2003-February/217419.html>
51. Fredrik L. [Python-Dev] Please Vote–Generator Comprehensions. <https://mail.python.org/pipermail/python-dev/2002-March/020717.html>
52. Josiah C. [Python-Dev] PEP 326 (quick location possibility) <https://mail.python.org/pipermail/python-dev/2004-January/042300.html>
53. Python Mailing List Discussions: PEP 326 Rejection—PEP 326 (quick location possibility). <https://mail.python.org/pipermail/python-dev/2004-January/042306.html>
54. Python Mailing List Discussions: PEP 308 Acceptance—Conditional Expression Resolution. <https://mail.python.org/pipermail/python-dev/2005-September/056846.html>
55. Fleiss Joseph L, Bruce L, Cho PM. *Statistical Methods for Rates and Proportions*. John Wiley & Sons; 2013.
56. Hallgren KA. Computing inter-rater reliability for observational data: an overview and tutorial. *Tutor Quant Methods Psychol*. 2012;8(1):23–34.
57. Paul S. Rationalising division (was: Re: [Fwd: Comment on PEP-0238]). <https://mail.python.org/pipermail/python-list/2001-July/066577.html>
58. Pankajeshwara S, Roy SBT, Nigel S. *Boundary Spanners in Open Source Software Development: A Study of Python Email Archives*. IEEE; 2017:308–317.
59. OpenJDK . mail.openjdk.java.net: Mailing Lists <http://mail.openjdk.java.net/2020>
60. Bitcoin . Bitcoin Bips <https://github.com/bitcoin/bips/2020>
61. Debain. DEP–Debian Enhancement Proposals <https://dep-team.pages.debian.net/2020>
62. Brian F. Open source software adoption: anatomy of success and failure. *Int J Open Source Software Proces*. 2009;1(1):1–23.
63. Sherlock L, Anne P, MacDonell Stephen G Supporting agile team composition: A prototype tool for identifying personality (In) compatibilities. In IEEE Computer Society, 2009, 66–73 .
64. Jake E. A panel with the new Python steering council <https://lwn.net/Articles/788404/2019>
65. OpenJDK . JEP 1: JDK Enhancement-Proposal & Roadmap Process. <https://openjdk.java.net/jeps/12020>
66. Python. PEP 1 <https://www.python.org/dev/peps/pep-00012019>
67. Mehwish R, Roxana G. *Another look at causality: Discovering scenario-specific contingency relationships with no supervision*. IEEE; 2010:361–368.
68. Mieliestronk. Mieliestronk's list of more than 58000 English words <http://www.mieliestronk.com/wordlist.html>

How to cite this article: Nand Sharma P, Tony Roy Savarimuthu B, Stanger N. Unearthing open source decision-making processes: A case study of python enhancement proposals. *Softw Pract Exper*. 2022;52(10):2312–2346. doi: 10.1002/spe.3128

APPENDIX A. DECISION MAKING IN PYTHON OSSD COMMUNITY

The Python community has publicised the set of states a PEP document goes through in PEP 1.¹⁷ Figure A1 describes PEP 1 and the main states within the process used to decide on the outcome of PEPs. The figure shows the high-level process; there are eight possible states that a proposal can go through before being made *Final*, *Rejected* or *Withdrawn*. *Accepted* PEPs that propose changes to Python itself are marked as ‘final’ once implementation of the changes is completed, thus freezing the PEP and prohibiting further changes. The only way to change a finalised PEP is to create a new PEP that supersedes it. *Accepted* Process PEPs describe processes and conventions (such as *voting*): however, they are often “living documents” that can be changed as needed. These are marked as *Active* instead of *Final* and remain active forever.

A.1 TOPIC MODELLING FOR EXTRACTING PYTHON DM CONCEPTS

This section outlines the details of the Topic Modelling (TM) approach mentioned in Section 3, for extracting DM concepts from the remaining dataset (after manually analyzing 30 PEPs).

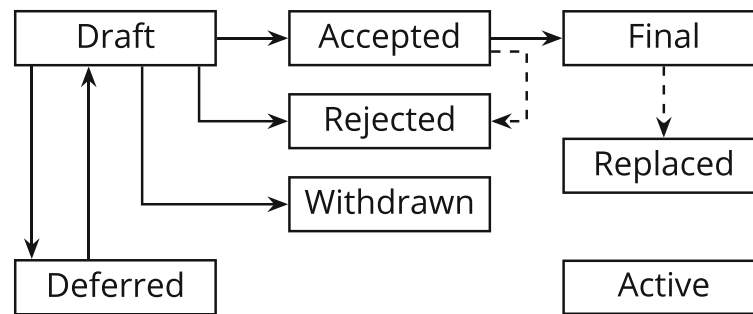


FIGURE A1 Python enhancement proposal decision-making process.^{7,66}

We implemented TM as part of our exploratory study as our intuition was that there would be at least some cases where events describing a particular topic-specific scenario (DM concept in our case) tend to be dependent on each other and may exist in the same sentence. Therefore, TM can be used to identify these terms (e.g., a sentence about the BDFL making a pronouncement on a PEP can refer to the *BDFL Pronouncement*, *reject*, *no support*, etc.). Using these terms we can also extract more sentences from the repository to further understand DM on PEPs. This automated approach is to capture both PEP sub-states that we do not know (for example, *BDFL Pronouncement* in PEPs). We tested TM on the 30 manually examined PEPs (outlined in Section 3) first, before moving onto the entire PEP repository.

There are some challenges to applying TM in software repositories, which related to extracting information from the email message body. The main challenge is that there are too many programming terms within the email messages. These terms show up in TM results. Sample coding terms include, “while”, “for”, “function”, “disk”, and “network”. At first, we attempted to add these coding terms to the stopword list and conducted the TM again. After a large number of attempts and re-runs, it was obvious that this was a time-consuming task. A better solution had to be created, which we describe below.

Essentially, we aim to identify a list of contingency relations between two events: the various DM states (and sub-states) and the cause behind the transition between states. A contingency relationship is a relationship between two events, one being “contingent” or a consequence of the other event. An example contingency relationship in the Python DM is the *Accepted* state which is a direct result of the *consensus* sub-state. There are some studies reported in the literature which aim to identify contingency relations from text automatically. In this part of our study, we borrow some steps from the methodology presented by Riaz et al.⁶⁷ Our initial steps are quite elaborate as we intend to extract contingency relations from OSSD email messages.

Step a—Identifying sentences containing the word “PEP”: For all email messages belonging to each PEP, we extracted all sentences which have the term “PEP”. We believe that the sub-states would be stated in these sentences, or the sentences before and after these sentences. We therefore extracted the previous, current and next paragraphs of this sentence and stored these into a database table. Our intuition is through this approach some candidate sentences and terms were bound to be found.

Step b—Removing coding terms: Since there were numerous coding terms in the email messages, instead of iteratively removing these terms, we considered only words in the English dictionary and left out the rest. We used a good source⁶⁸ for the set of all English words. This list is more comprehensive we found and the list also contained each term’s lemmatised version. The ignored terms were stored in another table for later checking if they included any essential terms related to DM concepts.

Step c—Configuring TM parameters: Afterwards, we ran the TM on these extracted terms. We began TM with a configuration of 20 topics and 20 terms for each topic, and then tested with little variations in these values. The main criteria was that the co-occurring terms within the same topic in the TM results should be terms used by members to convey the same DM concept, or main state. For instance, terms such as “pronouncement”, “consensus”, “support” in the same topic denotes the *Accepted* main state. The objective of this TM approach is that, if rows in TM results include several terms associated with the same DM concept, a DM concept as a label can be assigned for each row. We achieved the best TM results with 25 topics and 15 terms. On these final results, we labelled assigned a DM concept as a label for each topic.

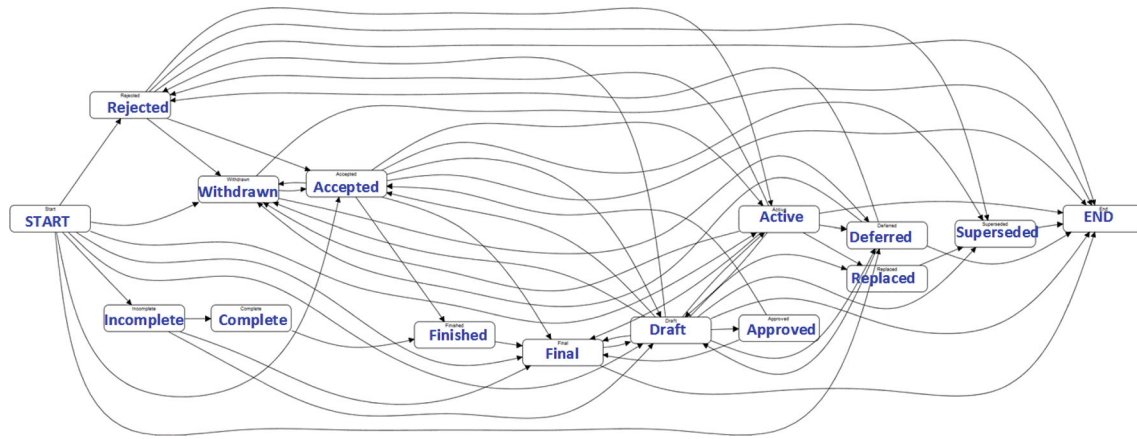


FIGURE A2 Additional states beyond those described in PEP 1⁸

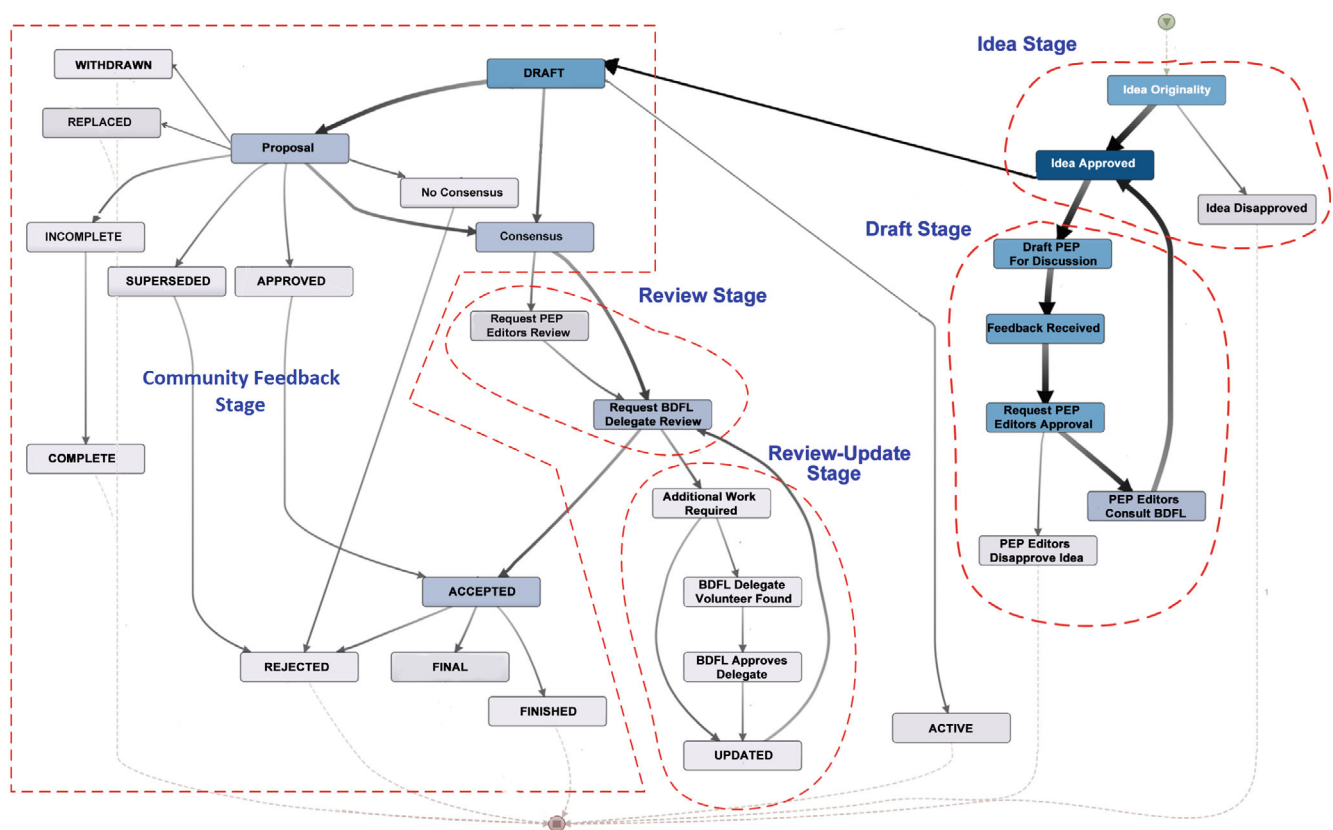


FIGURE A3 Python DM process, based on the textual description in PEP 1¹⁶

Table A4 highlights terms that frequently occurred when a DM concept (in the last column) is stated in sentences for all PEP messages in the repository. The co-occurring terms in each topic which reflect DM concepts, found through our manual study, are highlighted in yellow. As one can see from the TM results, we have been able to identify several terms used by developers in conveying the new DM concepts that we have identified through our previous analysis. **Thus, the results of TM did not reveal any new DM concepts and terms beyond those found through our manual analysis.**

Discussion of topic modelling results: The TM approach provides a technique to automatically capture a variety of DM concepts and related terms, without requiring the expansive manual analysis process. Future work can enhance the pre-processing step by incorporating the Named Entity Recognition (NER) method into the application. This way,

Topic	Co-occurring Terms in Topic	Label
0.	feedback (17), receive (9), status (8), date (8), implement (7), comment (5), experience (3), provisional (2), title (2), wink (2), edge (1), misrepresentation (1), pick (1), nit (1), invoke (1).	Feedback
1.	pep (12), interest (9), support (9), generate (8), defer (8), fail (6), significant (5), community (5), change (5), lack (4), finish (4), lukewarm (3), clarity (3), year (3), proposal (3),	Support
2.	pep (44), accept (21), acceptance (11), happy (9), propose (6), arithmetic (5), comment (5), change (5), czar (4), approval (4), mind (4), miss (3), conjunction (2), extended (2), announce (2).	Accept
3.	pep (22), poll (12), feedback (11), latest (8), informal (7), round (6), push (6), sufficiently (5), idea (5), confident (4), friendly (4), controversy (4), remain (4), embed (3), preferred (3).	Poll
4.	decision (14), major (4), assign (4), benefit (3), leave (3), objection (3), resolve (3), accept (3), datum (2), vote (2), ruling (1), warning (1), usual (1), skim (1), roadblock (1).	Review
5.	pep (42), review (31), ready (16), final (6), change (5), latest (4), controversial (3), developer (3), minor (2), improvement (2), send (2), involve (2), feedback (2), occasion (1), meantime (1).	Review
6.	pep (47), withdraw (22), author (11), idea (9), request (8), emerge (7), absolutely (4), exist (4), nice (4), pre (3), migration (3), detailed (3), useless (3), proven (3), afraid (3).	Withdraw
7.	pep (18), change (11), pronouncement (10), discussion (9), consensus (8), view (6), ready (5), basically (5), final (4), remain (4), concern (4), outstanding (3), decision (3), situation (3), meet (3).	Consensus
8.	decide (11), consensus (9), pronouncement (8), obsolete (8), resolve (7), commit (5), earlier (5), hope (5), rough (5), concur (4), immediately (3), topic (3), discussion (3), evaluate (2), explanation (2).	Consensus
9.	vote (13), positive (12), pep (12), response (11), show (5), community (5), negative (4), strongly (3), opinion (3), popular (2), feel (2), preference (2), complementary (2), election (2), precise (1).	Vote
10.	pep (17), reject (11), agree (10), disagree (8), entire (4), eventually (3), agreement (2), amount (2), step (2), turn (2), replacement (2), ahead (1), comparable (1), kindly (1), fan (1).	Reject
11.	pep (16), sprint (5), acceptance (4), discussion (4), great (3), aware (3), produce (3), exist (3), board (3), consensus (3), congratulation (2), fruitful (2), unary (2), major (2), community (2).	Accept
12.	pep (53), round (24), security (12), enhancement (9), preserve (6), dependency (6), identification (5), delegate (4), feel (3), wrong (3), safe (2), professional (2), centric (2), candidate (2), logical (2).	
13.	consensus (24), reach (10), alternative (9), majority (7), community (6), terminology (3), deem (3), die (3), adopt (3), user (3), lack (3), reduce (2), prevail (2), debate (2), unable (2).	Consensus
14.	change (14), consensus (13), idea (5), indentation (4), treat (3), official (3), significant (3), agree (3), huge (2), anonymous (2), wonderful (2), box (2), aspect (2), technical (2), century (2).	
15.	pep (48), defer (20), feedback (18), deferral (17), lack (15), cover (15), incorporate (15), interested (15), concept (14), effectively (14), collect (13), champion (13), exploration (13), sufficient (13), promote (11).	Deferred
16.	pep (61), author (22), withdraw (15), reject (13), supersede (8), support (7), external (6), user (6), modify (6), round (5), community (4), repository (2), determine (2), trunk (2), revision (2).	Withdraw
17.	pep (29), discussion (17), accept (17), discuss (8), pronouncement (7), hint (6), reason (6), recent (6), agree (5), suggest (3), reflect (3), virtually (2), protect (2), summit (2), accord (2).	Accept
18.	idea (13), consensus (8), generally (4), concern (4), team (3), track (3), chunk (2), blank (2), principle (2), reach (2), express (2), wild (1), sit (1), announce (1), unverifiable (1).	Idea
19.	idea (11), discussion (11), propose (10), pronounce (9), pep (9), year (7), disagreement (5), feedback (4), formally (3), early (3), forget (3), attempt (2), alas (2), integrate (2), significant (2).	Idea
20.	consensus (27), reach (14), minimality (4), group (3), accord (2), approval (2), corporate (2), encumber (2), neglect (2), motivated (2), throat (2), relevant (2), focus (2), presentation (2), convincing (2).	Consensus
21.	proposal (14), objection (9), raise (8), bring (6), support (6), quick (4), poll (4), essay (3), develop (3), keynote (2), habit (2), reflect (2), user (2), presentation (2), validate (1).	Poll
22.	pep (20), approve (10), comment (10), state (6), objection (5), regard (3), invite (3), plan (2), shortly (2), substantive (2), highly (2), direct (2), assume (2), eliminate (2), revise (2).	Accept
23.	consensus (10), reach (5), accept (5), requirement (4), choice (4), inclusion (3), topic (3), moment (2), quickly (2), weird (2), grow (2), reconsider (2), explain (2), oppose (1), harder (1).	Consensus
24.	pep (70), reject (43), rejection (14), accept (11), favour (6), proposal (6), modify (5), speak (4), rename (4), decree (3), checkin (2), increase (2), suggestion (2), yesterday (2), overwhelming (2).	Reject

FIGURE A4 Topic modelling results on the Python email messages based on all previous. Current and next sentences with the term “PEP”

developer names and roles, for example, *BDFL*, *Guido*, *users*, *developers* and others may be automatically removed from the sentences, instead of specifying them as stop words.